

claude-windows / dbc4cdae-04f6-4d06-af90-397bbce3fc57

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\subagents\agent-a6a3f9dabc34e2ad2.jsonl`  
- SHA-256: `b828e70eb1bc58640d68b8a48f949c8976c8c3bc543822af632e04e6dd4f2c`  
- Source modified: `2026-07-06T20:17:17+00:00`  
- Imported at: `2026-07-08T15:59:36+00:00`  
- project: `subagents`  
- session_id: `dbc4cdae-04f6-4d06-af90-397bbce3fc57`
```

Transcript

1. user (2026-07-06T20:11:11.836Z)

Read-only investigation in the repo at C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer. I'm building a torch-2.x GPU-resident reimplementation of the WASB shuttle detector and need to (a) reproduce its inference exactly and (b) score it on golden videos with F1/tolF1. Do NOT modify code.

Answer these with file:line evidence and concrete values:

1. WASB inference internals (to reproduce under torch 2.x)

From backend/pipeline/detectors/wasb_infer.py and native_wasb_runner.py, detail the FULL inference path so I can rebuild it:

- Model construction + weight load: how HRNet is built, how `wasb\_badminton\_best.pth.tar` is loaded (the exact `torch.load` + `load\_state\_dict` calls, and what key holds the weights).
- Preprocessing: the exact network input resolution (`inp\_width`/`inp\_height` ? trace where these come from in the WASB model config), the normalization (mean/std? scale?), the color order (RGB/BGR), and the sliding-window stacking (`frames\_in` ? how many frames per window, how they're concatenated as model input).
- Forward + postprocess: how the heatmap output is decoded to an (x,y) shuttle position per frame (argmax? blob? threshold?), and how per-frame positions become the trajectory.
- Output format: what exactly the runner emits (per-frame x,y,visibility? a CSV?).

2. Golden eval harness ? how F1/tolF1 is computed

From run_eval.py, run_regression.py, backend/eval/* (esp. anything named fusion_golden, metrics, or scoring), and docs/GOLDEN_REGRESSION_FIXTURES.md and docs/EVALUATION.md:

- The exact F1 / tolF1 definition and the distance threshold (? in pixels) used. What is "tolF1" vs "F1"?
- The command(s) to run an eval on a single video and get F1 numbers. What inputs it needs (video + ground-truth label file).

3. Golden video + ground-truth label locations & format

- Where do the golden eval VIDEOS live (local paths and/or GCS URIs ? check config.json, eval_baselines/, run_eval.py, docs/DATA_IN_GCS.md, docs/GOLDEN_REGRESSION_FIXTURES.md)? List the actual file names / GCS paths if the code references them.
- Where are the GROUND-TRUTH labels (per-frame shuttle x,y,visibility)? What format (CSV columns)? How are video?label paired?
- Is there a small subset (2-3 clips) I can use for a quick numbers run? Name them + their GCS/local paths + resolution/fps if stated.

4. Baseline numbers to compare against

- What are the current/known WASB F1 numbers on this golden set (from eval_baselines/*.json or docs)? Give the file + values so I can sanity-check a reimplementation.

Return a concise, concrete report with exact file:line, the eval command, and the golden video + label paths. Flag anything you can't find as an open question.

2. assistant (2026-07-06T20:11:14.796Z)

I'll investigate this WASB shuttle detector reimplementation task. Let me start by exploring the repo structure and reading the key files.

3. user (2026-07-06T20:11:17.500Z)

backend\pipeline\detectors\wasb_infer.py

4. user (2026-07-06T20:11:17.912Z)

```
true
---TOP---
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
---LS---
total 722
drwxr-xr-x 1 avidu 197609      0 Jul  6 23:47 .
drwxr-xr-x 1 avidu 197609      0 Jul  5 22:11 ..
drwxr-xr-x 1 avidu 197609      0 Jun 25 23:31 .claude
-rw-r--r-- 1 avidu 197609 94208 Jul  6 23:47 .coverage
-rw-r--r-- 1 avidu 197609   611 Jun 11 10:30 .coveragerc
-rw-r--r-- 1 avidu 197609    69 Jun 10 23:25 .env
drwxr-xr-x 1 avidu 197609      0 Jul  7 00:30 .git
-rw-r--r-- 1 avidu 197609   947 Jun 25 16:24 .gitattributes
drwxr-xr-x 1 avidu 197609      0 Jul  2 08:13 .github
-rw-r--r-- 1 avidu 197609  1998 Jul  2 08:13 .gitignore
drwxr-xr-x 1 avidu 197609      0 Jun 11 10:42 .mypy_cache
drwxr-xr-x 1 avidu 197609      0 May 19 15:58 .pytest_cache
drwxr-xr-x 1 avidu 197609      0 Jun 11 10:32 .ruff_cache
drwxr-xr-x 1 avidu 197609      0 Jun 25 16:24 .vscode
drwxr-xr-x 1 avidu 197609      0 Jul  2 08:20 __pycache__
drwxr-xr-x 1 avidu 197609      0 Jul  4 00:22 artifacts
drwxr-xr-x 1 avidu 197609      0 Jul  5 19:07 backend
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 backend.db
-rw-r--r-- 1 avidu 197609  6619 Jul  2 11:50 CLAUDE.md
-rw-r--r-- 1 avidu 197609  2735 Jun 16 01:52 config.json
drwxr-xr-x 1 avidu 197609      0 Jun 25 16:24 deploy
drwxr-xr-x 1 avidu 197609      0 Jul  6 22:19 docs
drwxr-xr-x 1 avidu 197609      0 Jul  4 06:58 eval_baselines
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 indexer.db
-rw-r--r-- 1 avidu 197609  1999 Jun 25 16:24 mypy.ini
drwxr-xr-x 1 avidu 197609      0 Jul  6 23:47 output
-rw-r--r-- 1 avidu 197609  5413 Jul  4 13:58 pyproject.toml
-rw-r--r-- 1 avidu 197609   240 May 30 14:08 pytest.ini
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 rally_index.db
-rw-r--r-- 1 avidu 197609 35511 Jul  2 08:13 README.md
-rw-r--r-- 1 avidu 197609  1674 Jul  2 20:05 requirements.txt
-rw-r--r-- 1 avidu 197609   585 Jul  2 19:49 requirements-trackers.txt
-rw-r--r-- 1 avidu 197609   809 Jun 25 16:24 ruff.toml
-rwxr-xr-x 1 avidu 197609  2118 Jun 29 13:47 run_all_tests.py
-rw-r--r-- 1 avidu 197609  5739 Jun 29 13:47 run_eval.py
-rw-r--r-- 1 avidu 197609  4594 Jun 29 13:47 run_regression.py
drwxr-xr-x 1 avidu 197609      0 Jul  2 21:43 scratch
drwxr-xr-x 1 avidu 197609      0 Jul  5 19:39 scripts
-rw-r--r-- 1 avidu 197609  2410 Jul  4 06:58 SESSION_HANDOFF.md
drwxr-xr-x 1 avidu 197609      0 Jul  6 23:54 tests
drwxr-xr-x 1 avidu 197609      0 Jul  2 19:37 tools
drwxr-xr-x 1 avidu 197609      0 Jun 10 15:57 training
```

5. user (2026-07-06T20:11:18.004Z)

backend\pipeline\detectors\native_wasb_runner.py

6. user (2026-07-06T20:11:18.537Z)

run_eval.py

7. user (2026-07-06T20:11:18.627Z)

run_regression.py

8. assistant (2026-07-06T20:11:27.357Z)

Let me read the core files in parallel.

9. user (2026-07-06T20:11:29.148Z)

```
1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       `
```

```

51
52     CACHE_VERSION = 1
53     _DET_FILE = "det_raw.jsonl"
54     _MANIFEST_FILE = "manifest.json"
55
56
57     def _build_cfg(weights: str, sport: str, device: str = "cuda"):
58         """Compose the WASB eval config via Hydra, overriding model/weights/device.
59
60         ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
61         preserved),
62         ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
63         ``runner.gpus=[0]``;
64         cpu/mps run with no GPU list so the detector places tensors on ``device``.
65         """
66         from hydra import compose, initialize
67
68         gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
69         with initialize(config_path="configs", version_base=None):
70             cfg = compose(
71                 config_name="eval",
72                 overrides=[
73                     f"dataset={sport}",
74                     "model=wasb",
75                     f"detector.model_path={weights}",
76                     f"runner.device={device}",
77                     f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to
78                 ],
79             )
80             return cfg
81
82     def _frame_id(path: str) -> int:
83         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
84         180)."""
85         stem = osp.splitext(osp.basename(path))[0]
86         digits = "".join(ch for ch in stem if ch.isdigit())
87         return int(digits) if digits else -1
88
89     # ----- #
90     # Resumable detector cache (pure-Python, no torch/numPy ? unit-testable)
91     # ----- #
92     def _cache_paths(cache_dir: str):
93         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
94
95     def default_cache_dir(out_csv: str) -> str:
96         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
97         d = osp.dirname(osp.abspath(out_csv))
98         stem = osp.splitext(osp.basename(out_csv))[0]
99         return osp.join(d, stem + "_wasbcache")
100
101
102     def _atomic_write_text(path: str, text: str) -> None:
103         """Write then rename so a crash mid-write can't leave a half-written file."""
104         tmp = path + ".tmp"
105         with open(tmp, "w") as f:
106             f.write(text)
107             f.flush()
108             os.fsync(f.fileno())
109         os.replace(tmp, path)
110
111

```

```

112     def _det_plain(det) -> list:
113         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
114         xy = det["xy"]
115         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
116
117
118     def _dets_to_tracker(plain_list):
119         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
120         the tracker does vector math / np.linalg.norm on it)."""
121         import numpy as np
122
123         return [
124             {"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
125             for p in plain_list
126         ]
127
128
129     def write_manifest(cache_dir: str, manifest: dict) -> None:
130         _, mpath = _cache_paths(cache_dir)
131         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
132
133
134     def read_manifest(cache_dir: str):
135         _, mpath = _cache_paths(cache_dir)
136         if not osp.exists(mpath):
137             return None
138         try:
139             with open(mpath) as f:
140                 return json.load(f)
141         except (json.JSONDecodeError, OSError):
142             return None
143
144
145     def manifest_compatible(
146         manifest: dict,
147         *,
148         frames_dir: str,
149         weights: str,
150         sport: str,
151         frames_in: int,
152         frames_total: int,
153         device: str = "cuda",
154     ) -> bool:
155         """A cache is reusable only if the run that produced it matches this run's inputs.
156
157         ``device`` defaults to ``cuda`` so caches written before device-keying (no
158         field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
159         cache
160         (detections can differ numerically across devices), and vice-versa.
161
162         ``frames_dir`` is the already-canonical cache key the caller stored in the
163         manifest
164         (``run()`` passes its ``cache_key`` here, and writes the same value into the
165         manifest):
166         ``osp.abspath(frames_dir)`` for the frames-on-disk path, or the synthetic
167         ``video:<abspath>``
168         key for the streaming path. We therefore compare it VERBATIM ? do NOT re-``abspath``
169         it here:
170         ``osp.abspath("video:/x.mp4")`` is not recognised as absolute and gets the CWD
171         prepended,
172         so a streaming cache would never match itself and every re-run would recompute
173         (issue #348).

```

```

167     """
168     if not manifest or manifest.get("version") != CACHE_VERSION:
169         return False
170     return (
171         manifest.get("frames_dir") == frames_dir
172         and manifest.get("weights") == weights
173         and manifest.get("sport") == sport
174         and manifest.get("frames_in") == frames_in
175         and manifest.get("frames_total") == frames_total
176         and manifest.get("device", "cuda") == device
177     )
178
179
180 def reset_cache(cache_dir: str) -> None:
181     """Drop any existing cache so the next run starts clean."""
182     det_jsonl, mpath = _cache_paths(cache_dir)
183     for p in (det_jsonl, mpath):
184         if osp.exists(p):
185             os.remove(p)
186
187
188 def load_and_clean_cache(cache_dir: str):
189     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
190     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
191     prefix so a trailing half-written line from a crash can't corrupt future appends.
192
193     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
194     [x, y, score, scale] candidates (accumulated across every window the frame is in,
195     in window order ? identical to a non-resumed run).
196     """
197     det_jsonl, _ = _cache_paths(cache_dir)
198     det_by_fid = defaultdict(list)
199     valid: list = []
200     if osp.exists(det_jsonl):
201         with open(det_jsonl, "r") as f:
202             for line in f:
203                 s = line.strip()
204                 if not s:
205                     continue
206                 try:
207                     obj = json.loads(s)
208                 except json.JSONDecodeError:
209                     break # truncated trailing line (crash mid-flush)
210                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
211                     break
212                 valid.append(s)
213                 for fid, plain in obj["f"]:
214                     det_by_fid[int(fid)].extend([list(p) for p in plain])
215                 # Guarantee a clean append boundary by rewriting only the good prefix.
216                 _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
217     return len(valid), det_by_fid
218
219
220 def _append_windows(fh, objs) -> None:
221     """Append window records as JSONL and durably flush (bounded loss on crash)."""
222     for o in objs:
223         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
224     fh.flush()
225     os.fsync(fh.fileno())
226
227
228 def _atomic_write_trajectory(out_csv: str, rows) -> None:
229     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
230     tmp = out_csv + ".tmp"
231     with open(tmp, "w", newline="") as f:

```

```

232         w = csv.writer(f)
233         w.writerow(["Frame", "Visibility", "X", "Y"])
234         w.writerows(rows)
235         f.flush()
236         os.fsync(f.fileno())
237     os.replace(tmp, out_csv)
238
239
240     # ----- #
241     # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
242     # ----- #
243     _STREAM_FRAME_FMT = "{:08d}.png"
244
245
246     def synthetic_frame_path(index: int) -> str:
247         """Stable, index-encoding frame name used by the streaming path.
248
249         It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
250         downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
251         frames came from disk or from the in-memory stream. The streaming image loader
252         inverts this name back to an index to fetch the decoded frame.
253         """
254         return _STREAM_FRAME_FMT.format(int(index))
255
256
257     def build_windows(frames: list, frames_in: int):
258         """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
259         + checkpoint). Pure list math, identical for the disk and streaming paths.
260
261         Returns a list of windows, each a list of ``frames_in`` consecutive paths
262         (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
263         caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
264         ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
265         """
266         return [frames[i : i + frames_in] for i in range(len(frames) - frames_in + 1)]
267
268
269     class SequentialFrameStore:
270         """In-memory sliding buffer over a forward-only frame reader, sized for the
271         detector's sliding-window access pattern (peak O(window) frames, not O(video)).
272
273         The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
274         ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
275         in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
276         start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
277         boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
278         far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
279         earliest index any not-yet-finished window can still ask for) and decode each source
280         frame exactly once via the forward-only ``reader(index)``.
281         """
282
283         def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
284             self._reader = reader
285             self._total = int(total)
286             self._window = max(1, int(window))
287             self._buf: dict = {}
288             # On a resumed run the detector's first needed frame is `start_index`; begin
289             # pulling there so the reader can skip (seek past) the already-cached prefix
290             # instead of decoding 0..start_index-1 just to throw them away.
291             self._next = int(start_index) # next index not yet pulled from the reader
292             self._max_seen = int(start_index) - 1
293             self._floor = int(start_index) # lowest index we still keep (never evict below)
294
295         def get(self, index: int):
296             index = int(index)

```

```

297         if index < self._floor:
298             raise KeyError(
299                 f"frame {index} already evicted (below floor {self._floor}); the access
300 "
301                 f"pattern must stay within a window of the highest frame seen"
302             )
303         if index >= self._total:
304             raise KeyError(f"frame {index} out of range (total {self._total})")
305         # Pull forward from the reader until the requested index is buffered.
306         while self._next <= index:
307             self._buf[self._next] = self._reader(self._next)
308             self._next += 1
309         if index > self._max_seen:
310             self._max_seen = index
311         # Evict frames older than the earliest a still-running window could re-request:
312         # once we've seen index `m`, no future window starts before `m - (window-1)`.
313         new_floor = max(self._floor, self._max_seen - (self._window - 1))
314         for k in range(self._floor, new_floor):
315             self._buf.pop(k, None)
316         self._floor = new_floor
317         return self._buf[index]
318
319     def _index_from_synthetic_path(path: str) -> int:
320         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
321
322         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
323         in lockstep with the on-disk naming scheme.
324         """
325         return _frame_id(path)
326
327     def _bgr_to_pil_rgb(frame_bgr):
328         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
329 EXACT
330         PIL RGB image WASB's disk path produces.
331
332         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
333 Image.open(PNG).convert('RGB')``;
334         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
335 COLOR_BGR2RGB)``
336         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
337 streamed
338         frame indistinguishable from the on-disk one to everything downstream.
339         """
340         import cv2
341         from PIL import Image
342
343         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
344
345     def _make_streamed_read_image(store, real_read_image):
346         """Build the ``read_image`` replacement used during a streaming detector pass.
347
348         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
349 image,
350         matching ``read_image``'s real return type); any path that doesn't parse to an index
351 falls
352         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
353 testable
354         without the WSL-only ``dataloaders`` package.
355         """
356
357     def _read_image(path, *args, **kwargs):
358         idx = _index_from_synthetic_path(path)

```

```

354         if idx < 0:
355             return real_read_image(path, *args, **kwargs)
356         return _bgr_to_pil_rgb(store.get(idx))
357
358     return _read_image
359
360
361     @contextmanager
362     def _streaming_read_image_patch(
363         frame_loader, total: int, window: int = 1, start_index: int = 0
364     ):
365         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
366         decoded frames during the detector pass, byte-for-byte as it would over real PNGs.
367
368         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
369         ``cv2.imread``. ``read_image`` (``utils.utils``) does
370         ``Image.open(path).convert('RGB')``,
371         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
372         feed it a PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
373         byte-identical to the disk round-trip, and the shim never touches the filesystem so
374         the ``osp.exists`` guard is sidestepped for synthetic stream paths.
375
376         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
377         actually calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
378         in that module's namespace; patching ``utils.read_image`` would NOT rebind the already-
379         imported reference).
380
381         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
382         wrapped in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
383         on a resume we start at ``start_index`` so the decoder seeks past already-cached windows.
384         The original reader is restored on exit.
385         """
386         from dataloaders import dataset_loader as _dl
387
388         store = SequentialFrameStore(
389             frame_loader, total, window=window, start_index=start_index
390         )
391         real_read_image = _dl.read_image
392         # Rebind read_image in the consuming module for the duration of the detector pass;
393         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
394         B010-clean.)
395         _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
396         ignore[assignment]
397         try:
398             yield store
399         finally:
400             _dl.read_image = real_read_image # type: ignore[assignment]
401
402     # ----- #
403     # Inference
404     # ----- #
405     class _PrefetchIter:
406         """Bounded producer-thread wrapper over a batch iterator (the num_workers=0
407         DataLoader).
408

```

```

406 In streaming mode the DataLoader must stay single-process (the in-memory frame store
+
407 the cv2.imread shim live in this process and cannot be pickled to workers), so the
408 CPU-side batch assembly (decode + PIL + ToTensor) SERIALIZES with GPU inference ?
409 measured on the first real L4 serving run (2026-07-03) this left the GPU <1%
utilized.
410 A single producer THREAD fixes the overlap without multiprocessing: cv2/PIL/numpy
411 release the GIL for the heavy parts, the store's forward-only access pattern is
412 preserved (one consumer thread iterating sequentially), and the bounded queue caps
413 memory at ``depth`` batches. A producer exception is re-raised in the consumer."""
414
415 _END = object()
416
417 def __init__(self, src, depth: int = 2):
418     import queue
419     import threading
420
421     self._q: "queue.Queue" = queue.Queue(maxsize=max(1, int(depth)))
422     self._exc: "BaseException | None" = None
423
424     def _fill() -> None:
425         try:
426             for item in src:
427                 self._q.put(item)
428             except BaseException as e: # noqa: BLE001 - surfaced in __next__
429                 self._exc = e
430             finally:
431                 self._q.put(self._END)
432
433     self._t = threading.Thread(target=_fill, name="wasb-prefetch", daemon=True)
434     self._t.start()
435
436 def __iter__(self):
437     return self
438
439 def __next__(self):
440     item = self._q.get()
441     if item is self._END:
442         if self._exc is not None:
443             raise self._exc
444         raise StopIteration
445     return item
446
447
448 def run(
449     frames,
450     weights: str,
451     out_csv: str,
452     sport: str = "badminton",
453     limit: int = 0,
454     batch_size: int = 8,
455     num_workers: int = 4,
456     log_every_batches: int = 50,
457     cache_dir: "str | None" = None,
458     flush_every: int = 1000,
459     fresh: bool = False,
460     frame_loader=None,
461     source_key: "str | None" = None,
462     device: str = "cuda",
463     prefetch_batches: int = 0,
464 ) -> str:
465     """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
466
467     Two equivalent ways to supply pixels (output is bit-identical between them):
468

```

```

469     * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
470     globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
``cv2.imread``.
471     * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
472     frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
473     pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
474     reader, which returns a PIL RGB image) over the DataLoader pass so every other
line of
475     ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
476     to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
477     ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
``num_workers=0``
478     (the in-process shim + buffer can't cross worker processes).
479
480     ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
481     frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
482     after reboot still matches.
483     """
484     import time
485
486     import torch
487     from dataloaders import build_img_transforms, build_seq_transforms
488     from dataloaders.dataset_loader import ImageDataset
489     from torch.utils.data import DataLoader
490
491     # NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside
the
492     # `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
suppresses
493     # the mypy false positive where the main env resolves `trackers` to the Roboflow
package
494     # (which has no `build_tracker`). There is NO runtime collision ? this runs in a
separate
495     # subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
dep.
496     from trackers import build_tracker # type: ignore[attr-defined]
497     from utils import Center
498
499     from detectors import build_detector
500
501     streaming = frame_loader is not None
502
503     cfg = _build_cfg(weights, sport, device)
504     frames_in = int(cfg["model"]["frames_in"])
505     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
506     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
507
508     if streaming:
509         # `frames` is already the ordered list of (synthetic) frame paths.
510         if not isinstance(frames, (list, tuple)):
511             raise SystemExit(
512                 "streaming mode requires `frames` to be a list of frame paths"
513             )
514         frames = list(frames)
515         frames_dir = source_key or "stream"
516     else:
517         frames_dir = frames
518         frames = sorted(
519             glob.glob(osp.join(frames_dir, "*.png"))
520             + glob.glob(osp.join(frames_dir, "*.jpg"))
521         )
522     if limit:
523         frames = frames[:limit]
524     if len(frames) < frames_in:

```

```

525         where = frames_dir if not streaming else "video stream"
526         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
527
528     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
529     samples = []
530     for window in build_windows(frames, frames_in):
531         annos = [
532             {"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
533             for p in window
534         ]
535         samples.append({"frames": window, "annos": annos})
536     windows_total = len(samples)
537
538     # ---- cache setup / resume decision ----- #
539     if cache_dir is None:
540         cache_dir = default_cache_dir(out_csv)
541     os.makedirs(cache_dir, exist_ok=True)
542     det_jsonl, _ = _cache_paths(cache_dir)
543     cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
544     manifest = None if fresh else read_manifest(cache_dir)
545     resume = manifest_compatible(
546         manifest or {},
547         frames_dir=cache_key,
548         weights=weights,
549         sport=sport,
550         frames_in=frames_in,
551         frames_total=len(frames),
552         device=device,
553     )
554     if resume:
555         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
556         logger.info(
557             f"resuming from cache: {windows_done}/{windows_total} windows already
detected"
558         )
559     else:
560         if manifest is not None:
561             logger.info("cache incompatible with this run -> starting fresh")
562             reset_cache(cache_dir)
563             windows_done, det_by_fid = 0, defaultdict(list)
564
565     base_manifest = {
566         "version": CACHE_VERSION,
567         "frames_dir": cache_key,
568         "weights": weights,
569         "sport": sport,
570         "frames_in": frames_in,
571         "frames_total": len(frames),
572         "device": device,
573         "windows_total": windows_total,
574         "windows_done": windows_done,
575     }
576     write_manifest(cache_dir, base_manifest)
577
578     # ---- detector pass over the un-cached windows ----- #
579     remaining = samples[windows_done:]
580     if remaining:
581         _, transform_test = build_img_transforms(cfg)
582         try:
583             _, seq_transform_test = build_seq_transforms(cfg)
584         except Exception:
585             seq_transform_test = None
586         ds = ImageDataset(
587             cfg,

```

```

588         remaining,
589         input_wh,
590         output_wh,
591         transform=transform_test,
592         seq_transform=seq_transform_test,
593         is_train=False,
594     )
595     # Streaming forces single-process loading: the in-memory frame store + the
596     # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
597     loader_workers = 0 if streaming else num_workers
598     loader = DataLoader(
599         ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
600     )
601     detector = build_detector(cfg)
602
603     from contextlib import ExitStack
604
605     t0 = time.time()
606     done = windows_done
607     win_idx = windows_done
608     log_every = max(1, log_every_batches)
609     flush_n = max(1, flush_every)
610     pending = []
611     logger.info(
612         f"detecting on {len(remaining)} remaining windows "
613         f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
614         f"source={'video-stream' if streaming else 'frames-dir'})..."
615     )
616     det_f = open(det_jsonl, "a")
617     try:
618         with ExitStack() as stack:
619             stack.enter_context(torch.no_grad())
620             # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
621             # in-memory decoded frame for that synthetic path; every other line of
622             # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
623             # The detector's first needed frame is `windows_done` (window i starts
at
624             # frame i), so prime the store there for a resume.
625             if streaming:
626                 stack.enter_context(
627                     _streaming_read_image_patch(
628                         frame_loader,
629                         len(frames),
630                         window=frames_in,
631                         start_index=windows_done,
632                     )
633                 )
634             # Overlap CPU batch assembly with GPU inference (streaming/workers=0
only ?
635             # a multi-worker DataLoader already overlaps via its worker processes).
636             batches = (
637                 _PrefetchIter(loader, depth=prefetch_batches)
638                 if prefetch_batches and loader_workers == 0
639                 else loader
640             )
641             for bi, batch in enumerate(batches):
642                 imgs, _hms, trans = batch[0], batch[1], batch[2]
643                 img_paths = [
644                     list(t) for t in batch[-1]
645                 ] # [frame_in][batch] -> path
646                 batch_results, _ = detector.run_tensor(imgs, trans)
647                 nb = imgs.shape[0]

```

```

648         for ib in range(nb):
649             frames_payload = []
650             for ie in sorted(batch_results[ib].keys()):
651                 p = img_paths[ie][ib]
652                 fid = _frame_id(p)
653                 plain = [_det_plain(d) for d in batch_results[ib][ie]]
654                 frames_payload.append([fid, plain])
655                 det_by_fid[fid].extend(plain)
656             pending.append({"w": win_idx, "f": frames_payload})
657             win_idx += 1
658         done += nb
659         if len(pending) >= flush_n or done >= windows_total:
660             _append_windows(det_f, pending)
661             pending = []
662             base_manifest["windows_done"] = done
663             write_manifest(cache_dir, base_manifest)
664         if (bi + 1) % log_every == 0 or done >= windows_total:
665             el = time.time() - t0
666             new_done = done - windows_done
667             rate = new_done / el if el > 0 else 0.0
668             eta = (windows_total - done) / rate if rate > 0 else 0.0
669             logger.info(
670                 f"{done}/{windows_total} "
671                 f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
672                 f"elapsed {el:.0f}s | ETA {eta:.0f}s"
673             )
674         if pending:
675             _append_windows(det_f, pending)
676             base_manifest["windows_done"] = done
677             write_manifest(cache_dir, base_manifest)
678     finally:
679         det_f.close()
680         logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
681     else:
682         logger.info("detector cache already complete -> tracker-only re-run")
683
684     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
685     tracker = build_tracker(cfg)
686     tracker.refresh()
687     rows = []
688     for fid in sorted(det_by_fid.keys()):
689         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
690         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
691
692     _atomic_write_trajectory(out_csv, rows)
693     visible = sum(1 for _, v, *_ in rows if v)
694     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
695     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
696     return out_csv
697
698
699 def extract_frames(video_path: str, frames_dir: str) -> str:
700     """Decode a video to PNG frames named by source frame index (00000000.png ...).
701
702     Idempotent: if the dir already holds the expected number of frames (matching the
703     container's reported frame count) we skip re-decoding ? re-extraction after a
704     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
705     extracting with ffmpeg on the host and passing --frames_dir.
706     """
707     import cv2
708
709     os.makedirs(frames_dir, exist_ok=True)
710     cap = cv2.VideoCapture(video_path)
711     if not cap.isOpened():

```

```

712         raise SystemExit(f"cannot open video: {video_path}")
713     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
714     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
715     if expected > 0 and existing >= expected:
716         cap.release()
717         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
718         return frames_dir
719     i = 0
720     while True:
721         ok, frame = cap.read()
722         if not ok:
723             break
724         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
725         i += 1
726     cap.release()
727     logger.info(f"extracted {i} frames -> {frames_dir}")
728     return frames_dir
729
730
731 def _probe_frame_count(video_path: str) -> int:
732     """Container-reported frame count via cv2 (0 if unknown). Used to size the
733     stream."""
734     import cv2
735     cap = cv2.VideoCapture(video_path)
736     if not cap.isOpened():
737         raise SystemExit(f"cannot open video: {video_path}")
738     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
739     cap.release()
740     return n
741
742
743 def make_video_frame_loader(video_path: str):
744     """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
745
746     ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
747     long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
748     detector's sliding-window order); it reads sequentially and only uses a frame-peek
749     to
750     skip forward when a *resume* starts past the current position. Each ``cap.read()``
751     returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
752     would
753     yield (PNG is lossless), so the detector sees identical pixels to the disk path.
754
755     Returns the count hint from the container so the caller can build the frame list;
756     ``release`` closes the capture.
757     """
758     import cv2
759     cap = cv2.VideoCapture(video_path)
760     if not cap.isOpened():
761         raise SystemExit(f"cannot open video: {video_path}")
762     count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
763     state = {"pos": 0} # next index cap.read() will return
764
765     def frame_loader(index: int):
766         index = int(index)
767         if index < state["pos"]:
768             raise RuntimeError(
769                 f"streaming decode is forward-only; got index {index} after
770                 {state['pos']}")
771         if index > state["pos"]:
772             # Forward skip only happens when resuming past cached windows. Seek once.
773             cap.set(cv2.CAP_PROP_POS_FRAMES, index)

```

```

773         state["pos"] = index
774     ok, frame = cap.read()
775     if not ok:
776         raise RuntimeError(f"failed to decode frame {index} from {video_path}")
777     state["pos"] += 1
778     return frame
779
780     def release():
781         cap.release()
782
783     return frame_loader, count_hint, release
784
785
786     def run_video_streaming(
787         video_path: str,
788         weights: str,
789         out_csv: str,
790         sport: str = "badminton",
791         limit: int = 0,
792         batch_size: int = 8,
793         log_every_batches: int = 50,
794         cache_dir: "str | None" = None,
795         flush_every: int = 1000,
796         fresh: bool = False,
797         device: str = "cuda",
798         prefetch_batches: int = 0,
799     ) -> str:
800         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
801         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
802         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
803         ``run``'s docstring for why.
804         """
805         frame_loader, count_hint, release = make_video_frame_loader(video_path)
806         try:
807             if count_hint <= 0:
808                 raise SystemExit(
809                     f"could not determine frame count for {video_path}; cannot stream "
810                     f"(fall back to frame extraction with --frames_dir)"
811                 )
812             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
813             source_key = "video:" + osp.abspath(video_path)
814             return run(
815                 frame_list,
816                 weights,
817                 out_csv,
818                 sport,
819                 limit,
820                 batch_size=batch_size,
821                 num_workers=0,
822                 log_every_batches=log_every_batches,
823                 cache_dir=cache_dir,
824                 flush_every=flush_every,
825                 fresh=fresh,
826                 frame_loader=frame_loader,
827                 source_key=source_key,
828                 device=device,
829                 prefetch_batches=prefetch_batches,
830             )
831         finally:
832             release()
833
834
835     def main():
836         ap = argparse.ArgumentParser()
837         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")

```

```

838     ap.add_argument(
839         "--video", help="video file; frames are extracted internally (in WSL)"
840     )
841     ap.add_argument(
842         "--frames_out_dir", help="where to extract frames when --video is used"
843     )
844     ap.add_argument(
845         "--stream-video",
846         action="store_true",
847         help="FAST PATH: decode --video on the fly and feed frames straight to "
848             "the detector (no PNG extraction). Output-equivalent to the disk path.",
849     )
850     ap.add_argument("--weights", required=True)
851     ap.add_argument("--out", required=True)
852     ap.add_argument("--sport", default="badminton")
853     ap.add_argument(
854         "--device",
855         default="cuda",
856         choices=["cuda", "mps", "cpu"],
857         help="torch device for the detector (default cuda = the WSL parity path)",
858     )
859     ap.add_argument(
860         "--limit", type=int, default=0, help="cap #frames (for quick tests)"
861     )
862     ap.add_argument("--batch-size", type=int, default=8)
863     ap.add_argument("--num-workers", type=int, default=4)
864     ap.add_argument(
865         "--prefetch-batches",
866         type=int,
867         default=0,
868         help="streaming mode: overlap CPU batch assembly with GPU inference via a "
869             "bounded producer thread of this queue depth (0 = off, the parity default)",
870     )
871     ap.add_argument(
872         "--log-every-batches", type=int, default=50, help="progress log cadence"
873     )
874     ap.add_argument(
875         "--cache-dir",
876         default=None,
877         help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
878     )
879     ap.add_argument(
880         "--flush-every",
881         type=int,
882         default=1000,
883         help="fsync the detector cache every N windows (bounds reboot loss)",
884     )
885     ap.add_argument(
886         "--fresh", action="store_true", help="ignore any existing cache and start over"
887     )
888     args = ap.parse_args()
889
890     if args.stream_video:
891         if not args.video:
892             raise SystemExit("--stream-video requires --video")
893         run_video_streaming(
894             args.video,
895             args.weights,
896             args.out,
897             args.sport,
898             args.limit,
899             batch_size=args.batch_size,
900             log_every_batches=args.log_every_batches,
901             cache_dir=args.cache_dir,
902             flush_every=args.flush_every,

```

```

903         fresh=args.fresh,
904         device=args.device,
905         prefetch_batches=args.prefetch_batches,
906     )
907     return
908
909     frames_dir = args.frames_dir
910     if args.video:
911         frames_dir = extract_frames(
912             args.video, args.frames_out_dir or (args.video + "_frames")
913         )
914     if not frames_dir:
915         raise SystemExit("provide --frames_dir or --video")
916     run(
917         frames_dir,
918         args.weights,
919         args.out,
920         args.sport,
921         args.limit,
922         batch_size=args.batch_size,
923         num_workers=args.num_workers,
924         log_every_batches=args.log_every_batches,
925         cache_dir=args.cache_dir,
926         flush_every=args.flush_every,
927         fresh=args.fresh,
928         device=args.device,
929     )
930
931
932 if __name__ == "__main__":
933     main()
934

```

10. user (2026-07-06T20:11:29.777Z)

```

1     """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3     The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4     "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5     It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6     Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7     that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8     see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10     * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14     (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15     strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16     * weights / repo / python / scratch come from the environment first
17     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18     ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}_{vid12}_wasb.csv``
21     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24     (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);

```

```

``detector_impl="native"``.
25
26     **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
runner
27     (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28     candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
29     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31     vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32     artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33     (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34     not the numbers.
35     """
36
37     import logging
38     import os
39     import shutil
40     import subprocess
41     from dataclasses import dataclass
42     from typing import List, Optional, Tuple
43
44     from backend.config.models import IndexingConfig
45     from backend.pipeline.detectors.base import DetectorRunner
46     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51     logger = logging.getLogger(__name__)
52
53     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54     # detectors/trackers/data loaders + find configs/), exactly as the WSL adapter does.
55     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58     # so the "cuda True/False" string they parse can never drift apart.
59     _CUDA_PROBE_CODE = (
60         "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61     )
62
63
64     @dataclass
65     class NativeWasbConfig:
66         """Resolved settings for a Linux-native WASB run.
67
68         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
69         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
70         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
71         """
72
73         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
74         python_bin: str = (
75             "python" # the `wasb` conda env python (invoked by path, no activate)
76         )
77         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)

```

```

78     sport: str = "badminton"
79     device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
80     scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
81     timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
82     keep_frames: bool = (
83         False # retain the decoded frame cache after success (debug only)
84     )
85     # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
86     # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
87     # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
88     stream_video: bool = True
89     # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
90     # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
91     batch_size: int = 0
92     prefetch_batches: int = 0
93
94     @classmethod
95     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
96         if isinstance(idx_cfg, dict):
97             idx_cfg = IndexingConfig(**idx_cfg)
98         w = idx_cfg.wasb
99         env = os.environ.get
100         # stream_video is tri-state in config: None/absent => native default (stream);
an
101         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
102         sv = w.stream_video
103         return cls(
104             repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
105             python_bin=env("WASB_PYTHON") or w.python_bin,
106             weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
107             sport=w.sport,
108             device=env("WASB_DEVICE") or w.device,
109             scratch_dir=env("WASB_SCRATCH")
110             or env("RALLY_SCRATCH")
111             or env("TMPDIR")
112             or "",
113             timeout_sec=w.timeout_sec,
114             keep_frames=w.keep_frames,
115             stream_video=(True if sv is None else bool(sv)),
116             batch_size=int(w.batch_size or 0),
117             prefetch_batches=int(w.prefetch_batches or 0),
118         )
119
120
121     class NativeWasbRunner(DetectorRunner):
122         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
123
124         def __init__(self, cfg: NativeWasbConfig):
125             self.cfg = cfg
126             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
127             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
128             self._cuda_available: Optional[bool] = None
129
130             # --- low-level native exec (the single place tests patch)
-----
131             def _run(
132                 self, argv: List[str], cwd: Optional[str] = None
133             ) -> subprocess.CompletedProcess:

```

```

134         logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
135         return subprocess.run(
136             argv,
137             cwd=cwd,
138             capture_output=True,
139             text=True,
140             timeout=(self.cfg.timeout_sec or None),
141         )
142
143     # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
144     def _probe_cuda(self) -> bool:
145         """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
146         (missing python / timeout / import error) is treated as 'no CUDA'."""
147         try:
148             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
149         except (FileNotFoundError, subprocess.TimeoutExpired):
150             return False
151         return res.returncode == 0 and "cuda True" in (res.stdout or "")
152
153     def _cuda_is_available(self) -> bool:
154         """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
155         if self._cuda_available is None:
156             self._cuda_available = self._probe_cuda()
157         return self._cuda_available
158
159     def _effective_device(self) -> str:
160         """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
161         probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
162         if self.cfg.device == AUTO:
163             return DeviceContext(AUTO, self._cuda_is_available()).effective
164         return self.cfg.device
165
166     def _repo_src(self) -> str:
167         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
168
169     def _copy_infer_script(self) -> bool:
170         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
171         repo_src = self._repo_src()
172         try:
173             os.makedirs(repo_src, exist_ok=True)
174             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
175             return True
176         except OSError as e:
177             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
178             return False
179
180     def _rm_rf(self, path: Optional[str]) -> None:
181         """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
182         if path:
183             shutil.rmtree(path, ignore_errors=True)
184
185     def healthcheck(self) -> Tuple[bool, str]:
186         """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
187         if not self.cfg.weights_path:
188             return False, (
189                 "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path)."
```

```

190         )
191         weights = os.path.expanduser(self.cfg.weights_path)
192         if not os.path.exists(weights):
193             return False, f"WASB weights not found at {weights}"
194         repo_src = self._repo_src()
195         if not os.path.isdir(repo_src):
196             return False, (
197                 f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
198                 f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
199         )
200         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
201         # argparse rc=2 from wasb_infer.py at run time.
202         if self.cfg.device not in VALID_DEVICES:
203             return False, (
204                 f"unknown device {self.cfg.device!r} ? valid: {'',
205                 '.join(VALID_DEVICES)})."
206             )
207         try:
208             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
209         except FileNotFoundError:
210             return (
211                 False,
212                 f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
213             )
214         except subprocess.TimeoutExpired:
215             return False, "native torch healthcheck timed out."
216         out = (res.stdout or "").strip()
217         if res.returncode != 0:
218             return (
219                 False,
220                 f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
221                 out).strip()}",
222             )
223         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
224         subprocess).
225         self._cuda_available = "cuda True" in out
226         ctx = DeviceContext(self.cfg.device, self._cuda_available)
227         if ctx.cuda_required_but_missing:
228             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
229             # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
230             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
231         if ctx.fell_back:
232             logger.warning(
233                 "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
234                 "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.",
235                 out,
236             )
237         return True, out
238
239     def run_predict(
240         self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
241     ) -> Optional[str]:
242         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
243         None.
244
245         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
246         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
247         the
248         same downstream contract, so the parity comparison is apples-to-apples.
249         """
250         output_dir = os.path.abspath(output_win_dir)

```

```

246         os.makedirs(output_dir, exist_ok=True)
247         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
248         norm = str(video_win_path).replace("\\", "/")
249         stem, _ext = os.path.splitext(os.path.basename(norm))
250         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
251         key = f"{stem}_{video_id[:12]}" if video_id else (stem or "wasb")
252         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
253
254         if not self._copy_infer_script():
255             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
256             return None
257
258         weights = os.path.expanduser(self.cfg.weights_path)
259         argv = [
260             self.cfg.python_bin,
261             "wasb_infer.py",
262             "--video",
263             os.path.abspath(str(video_win_path)),
264         ]
265         frames_dir: Optional[str] = None
266         if self.cfg.stream_video:
267             # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
268             argv.append("--stream-video")
269         else:
270             scratch = self.cfg.scratch_dir or output_dir
271             frames_dir = os.path.join(scratch, f"{key}_frames")
272             argv += ["--frames_out_dir", frames_dir]
273         # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
274         # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
275         device = self._effective_device()
276         argv += [
277             "--weights",
278             weights,
279             "--sport",
280             self.cfg.sport,
281             "--device",
282             device,
283             "--out",
284             out_csv,
285         ]
286         # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
287         # byte-parity guarantee) is unchanged when the knobs are unset.
288         if self.cfg.batch_size:
289             argv += ["--batch-size", str(self.cfg.batch_size)]
290         if self.cfg.prefetch_batches:
291             argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
292
293         logger.info(
294             "Running native WASB inference (device=%s%s) on %s ...",
295             device,
296             " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
297             os.path.basename(norm),
298         )
299         try:
300             res = self._run(argv, cwd=self._repo_src())
301         except FileNotFoundError:
302             logger.error(
303                 "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
304             )
305             return None

```

```

306         except subprocess.TimeoutExpired:
307             logger.error(
308                 "WASB native inference timed out after %ss.", self.cfg.timeout_sec
309             )
310             return None
311         if res.returncode != 0:
312             logger.error(
313                 "WASB native inference failed:\n%s",
314                 (res.stderr or res.stdout or "")[-2000:],
315             )
316             return None
317         if not os.path.exists(out_csv):
318             logger.error(
319                 "WASB native finished but expected CSV not found at %s", out_csv
320             )
321             return None
322         logger.info("WASB trajectory CSV: %s", out_csv)
323
324         # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
325         # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
326         # (Streaming never materializes frames, so there is nothing to drop on that
path.)
327         if not self.cfg.keep_frames and frames_dir:
328             logger.info(
329                 "Cache hygiene: removing frame cache for %s "
330                 "(set indexing.wasb.keep_frames=true to retain).",
331                 key,
332             )
333             self._rm_rf(frames_dir)
334         return out_csv
335
336         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
337         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
338         trajectory_to_action_windows = staticmethod(
339             TrackNetRunner.trajectory_to_action_windows
340         )
341

```

11. user (2026-07-06T20:11:30.252Z)

```

1     """CLI to evaluate detected rallies against hand-annotated ground truth.
2
3     Scores predictions already stored in the SQLite DB (run the pipeline first) ? no
4     API calls, fully deterministic. See docs/EVALUATION.md for the workflow.
5
6     Examples
7     -----
8     Create an empty annotation file to fill in:
9         python run_eval.py --scaffold --annotations rallies.csv
10
11     Score a video's detected rallies (identify it by file or by stored video_id):
12         python run_eval.py --video "C:/path/match.mp4" --annotations rallies.csv
13         python run_eval.py --video-id <md5> --annotations rallies.csv --stage both --iou 0.5
14
15     Show which rallies were missed / spurious, and dump a JSON report:
16         python run_eval.py --video-id <md5> --annotations rallies.csv --show-failures --json
report.json
17     """
18
19     import argparse
20     import json
21     import logging

```

```

22 import os
23 import sys
24
25 from backend.config import default_db_path
26 from backend.eval.annotations import load_annotations, write_scaffold
27 from backend.eval.harness import STAGES, evaluate, format_report_text, report_to_dict
28 from backend.infrastructure.database import Database
29 from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id
30
31 _EPILOG = """\
32 Prerequisite:
33     The harness scores predictions read from the SQLite DB ? it does NOT run the
34     pipeline. So you must first PROCESS the video (e.g. via run_process_and_stitch.py)
35     so its detections land in output/sports_indexer.db. Then point this tool at the
36     same video file (--video) or its stored MD5 (--video-id). If you get
37     "database not found" or zero predictions, the video hasn't been processed yet.
38
39 See docs/EVALUATION.md for the annotation format and how to read the output.
40 """
41
42
43 def build_parser() -> argparse.ArgumentParser:
44     p = argparse.ArgumentParser(
45         description="Evaluate detected rallies vs ground-truth annotations.",
46         epilog=_EPILOG,
47         formatter_class=argparse.RawDescriptionHelpFormatter,
48     )
49     p.add_argument(
50         "--annotations", help="Path to the ground-truth rally CSV (start,end per row)."
51     )
52     p.add_argument(
53         "--video", help="Path to the video file (its MD5 becomes the video_id)."
54     )
55     p.add_argument(
56         "--video-id", help="Stored video_id (MD5) to evaluate, instead of --video."
57     )
58     p.add_argument(
59         "--db", default=None, help="SQLite DB path (default: from config.json)."
60     )
61     p.add_argument(
62         "--stage",
63         choices=("candidates", "final", "both"),
64         default="both",
65         help="Which prediction stage(s) to score (default: both).",
66     )
67     p.add_argument(
68         "--detector",
69         default=None,
70         help="Only score candidates from this detector (e.g. yolo_hybrid,
71 tracknet_hybrid).",
72     )
73     p.add_argument(
74         "--iou", type=float, default=0.5, help="IoU match threshold (default: 0.5)."
75     )
76     p.add_argument(
77         "--pr-curve",
78         action="store_true",
79         help="Include an IoU sweep (0.1..0.9) in the report.",
80     )
81     p.add_argument(
82         "--show-failures",
83         action="store_true",
84         help="List missed (FN) and spurious (FP) rallies with timestamps.",
85     )
86     p.add_argument(

```

```

86         "--json",
87         dest="json_out",
88         default=None,
89         help="Write a JSON report to this path.",
90     )
91     p.add_argument(
92         "--scaffold",
93         action="store_true",
94         help="Write an empty annotation CSV to --annotations and exit.",
95     )
96     return p
97
98
99 def main(argv=None) -> int:
100     logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
101     args = build_parser().parse_args(argv)
102
103     # Scaffold mode: just write the template and exit.
104     if args.scaffold:
105         if not args.annotations:
106             print("ERROR: --scaffold requires --annotations <path>", file=sys.stderr)
107             return 2
108         write_scaffold(args.annotations)
109         print(f"Wrote empty annotation file: {args.annotations}")
110         return 0
111
112     if not args.annotations:
113         print("ERROR: --annotations is required (or use --scaffold).", file=sys.stderr)
114         return 2
115
116     # Resolve the video_id.
117     if args.video_id:
118         video_id = args.video_id
119     elif args.video:
120         if not os.path.exists(args.video):
121             print(f"ERROR: video not found: {args.video}", file=sys.stderr)
122             return 2
123         print("Computing video MD5...")
124         video_id = get_file_md5(args.video)
125         print(f"video_id = {video_id}")
126     else:
127         print("ERROR: provide --video <path> or --video-id <md5>.", file=sys.stderr)
128         return 2
129
130     db_path = args.db or default_db_path()
131     if not os.path.exists(db_path):
132         print(
133             f"ERROR: database not found at {db_path}. Run the pipeline first.",
134             file=sys.stderr,
135         )
136         return 2
137
138     gt = load_annotations(args.annotations)
139     if not gt:
140         print(
141             f"WARNING: no ground-truth rallies in {args.annotations}.", file=sys.stderr
142         )
143
144     stages = list(STAGES) if args.stage == "both" else [args.stage]
145     db = Database(db_path)
146     report = evaluate(
147         db,
148         video_id,
149         gt,
150         stages,

```

```

151         iou_threshold=args.iou,
152         detector=args.detector,
153         with_pr_curve=args.pr_curve,
154     )
155
156     print(format_report_text(report, show_failures=args.show_failures))
157
158     if args.json_out:
159         with open(args.json_out, "w") as f:
160             json.dump(report_to_dict(report), f, indent=2)
161             print(f"\nJSON report written to {args.json_out}")
162
163     return 0
164
165
166 if __name__ == "__main__":
167     raise SystemExit(main())
168

```

12. user (2026-07-06T20:11:30.642Z)

```

1     """CLI: regression-test the production tool against golden-set ground truth.
2
3     Obtains ground truth from an annotation provider (a tier), scores the video's
4     stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5     detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7     Examples
8     -----
9     Snapshot the current numbers as the baseline (first run / after a real improvement):
10         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
11         --update-baseline
12
13     Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
14         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
15
16     """
17
18     import argparse
19     import json
20     import logging
21     import os
22     import sys
23
24     from backend.annotations import annotation_registry
25     from backend.config import default_db_path
26     from backend.eval import regression
27     from backend.infrastructure.database import Database
28
29     def build_parser() -> argparse.ArgumentParser:
30         p = argparse.ArgumentParser(
31             description="Regression-test the tool vs golden-set ground truth."
32         )
33         p.add_argument(
34             "--video-id",
35             required=True,
36             help="Stored video_id (MD5) whose predictions to score.",
37         )
38         p.add_argument(
39             "--tier",
40             default="file",
41             help=f"Annotation provider tier. Available: {annotation_registry.list_available()}",
42         )
43         p.add_argument(

```

```

43         "--annotations",
44         help="video_ref for the provider (e.g. CSV path for the 'file' tier).",
45     )
46     p.add_argument("--stage", choices=("candidates", "final"), default="final")
47     p.add_argument(
48         "--detector", default=None, help="Restrict candidate scoring to one detector."
49     )
50     p.add_argument("--iou", type=float, default=0.5)
51     p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
52     p.add_argument("--db", default=None)
53     p.add_argument(
54         "--baseline",
55         default=regression.DEFAULT_BASELINE_PATH,
56         help="Baseline JSON path.",
57     )
58     p.add_argument(
59         "--update-baseline",
60         action="store_true",
61         help="Snapshot the current numbers as the new baseline instead of checking.",
62     )
63     p.add_argument(
64         "--allow-missing-baseline",
65         action="store_true",
66         help="Treat a missing baseline as a pass (exit 0) instead of a gate error (exit
67 3).",
68     )
69     p.add_argument("--json", dest="json_out", default=None)
70     return p
71
72 def main(argv=None) -> int:
73     logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
74     args = build_parser().parse_args(argv)
75
76     db_path = args.db or default_db_path()
77     if not os.path.exists(db_path):
78         print(
79             f"ERROR: database not found at {db_path}. Run the pipeline first.",
80             file=sys.stderr,
81         )
82         return 2
83     db = Database(db_path)
84
85     result = regression.regress_with_provider(
86         db,
87         video_id=args.video_id,
88         video_ref=args.annotations,
89         tier=args.tier,
90         stage=args.stage,
91         iou_threshold=args.iou,
92         detector=args.detector,
93         tolerance=args.tolerance,
94         baseline_path=args.baseline,
95     )
96
97     print(regression.format_result(result))
98
99     if args.update_baseline:
100         regression.save_baseline(
101             result.video_id,
102             result.stage,
103             result.precision,
104             result.recall,
105             result.f1,
106             path=args.baseline,

```

```

107         iou_threshold=result.iou_threshold,
108         detector=args.detector,
109         gt_fingerprint=result.gt_hash,
110     )
111     print(f"Baseline updated in {args.baseline}.")
112
113     if args.json_out:
114         with open(args.json_out, "w") as f:
115             json.dump(result.as_dict(), f, indent=2)
116             print(f"JSON written to {args.json_out}")
117
118     # Exit codes: 1 = regression, 3 = no baseline to compare (gate cannot pass), 0 = ok.
119     if result.regressed:
120         return 1
121     if (
122         not result.has_baseline
123         and not args.update_baseline
124         and not args.allow_missing_baseline
125     ):
126         print(
127             "ERROR: no baseline to compare against ? the gate cannot verify anything. "
128             "Snapshot one with --update-baseline (then commit it), or pass "
129             "--allow-missing-baseline to accept this run.",
130             file=sys.stderr,
131         )
132         return 3
133     return 0
134
135
136 if __name__ == "__main__":
137     raise SystemExit(main())
138

```

13. assistant (2026-07-06T20:11:36.981Z)

I have a strong grasp of the WASB inference path and that the run_eval/run_regression harness scores ****rally-level IoU**** (not per-pixel shuttle F1). The user is asking about per-pixel F1/tolF1 for shuttle detection ? that's likely a different eval harness. Let me look at the backend/eval directory, docs, config, and eval_baselines in parallel.

14. user (2026-07-06T20:11:41.723Z)

```

===== backend/eval =====
total 565
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 .
drwxr-xr-x 1 avidu 197609    0 Jul  5 19:07 ..
-rw-r--r-- 1 avidu 197609  455 May 30 14:08 __init__.py
drwxr-xr-x 1 avidu 197609    0 Jul  6 23:45 __pycache__
-rw-r--r-- 1 avidu 197609 24730 Jul  4 06:58 ablation.py
-rw-r--r-- 1 avidu 197609  8426 Jul  4 06:58 ablation_pdf.py
-rw-r--r-- 1 avidu 197609  2343 Jun 29 13:47 annotations.py
drwxr-xr-x 1 avidu 197609    0 Jun 29 13:47 audio
-rw-r--r-- 1 avidu 197609  4911 Jun 29 13:47 batch.py
-rw-r--r-- 1 avidu 197609 11827 Jul  4 06:58 calibrate_local.py
-rw-r--r-- 1 avidu 197609  5287 Jun 29 13:47 calibrate_wasb.py
-rw-r--r-- 1 avidu 197609  8288 Jun 29 13:47 calibration.py
-rw-r--r-- 1 avidu 197609  4762 Jun 29 13:47 classifier.py
-rw-r--r-- 1 avidu 197609  8970 Jul  4 06:58 distill_local.py
-rw-r--r-- 1 avidu 197609 12068 Jun 29 13:47 eval_partial_labels.py
-rw-r--r-- 1 avidu 197609  6905 Jun 29 13:47 eval_stats.py
-rw-r--r-- 1 avidu 197609 33755 Jun 29 13:47 experiment.py
-rw-r--r-- 1 avidu 197609 11899 Jun 29 13:47 export_wasb_segments.py
-rw-r--r-- 1 avidu 197609  8427 Jul  4 06:58 fusion_audio.py
-rw-r--r-- 1 avidu 197609  5163 Jun 29 13:47 fusion_compare.py
-rw-r--r-- 1 avidu 197609 19965 Jul  4 06:58 fusion_golden.py

```

```
-rw-r--r-- 1 avidu 197609 13922 Jun 25 22:09 gemini_refine.py
-rw-r--r-- 1 avidu 197609 44366 Jun 29 13:47 golden_fixtures.py
-rw-r--r-- 1 avidu 197609 14201 Jul  4 06:58 golden_manifest.py
-rw-r--r-- 1 avidu 197609 20260 Jun 29 13:47 golden_real_fixtures.py
-rw-r--r-- 1 avidu 197609 4772 Jun 29 13:47 gt_loader.py
-rw-r--r-- 1 avidu 197609 5636 Jun 29 13:47 harness.py
-rw-r--r-- 1 avidu 197609 5990 Jun 29 13:47 metrics.py
-rw-r--r-- 1 avidu 197609 8117 Jun 29 13:47 per_user_eval.py
-rw-r--r-- 1 avidu 197609 9262 Jun 29 13:47 per_user_gate.py
-rw-r--r-- 1 avidu 197609 14152 Jun 29 13:47 per_user_regression.py
-rw-r--r-- 1 avidu 197609 10359 Jun 29 13:47 promotion.py
-rw-r--r-- 1 avidu 197609 36192 Jul  4 06:58 rally_seg_eval.py
-rw-r--r-- 1 avidu 197609 7924 Jul  4 06:58 rally_seq_proto.py
-rw-r--r-- 1 avidu 197609 9356 Jun 29 13:47 regression.py
-rw-r--r-- 1 avidu 197609 6729 Jun 29 13:47 score_calibration.py
-rw-r--r-- 1 avidu 197609 10702 Jun 29 13:47 segmentation_metrics.py
-rw-r--r-- 1 avidu 197609 8680 Jul  4 06:58 serve_contrast.py
-rw-r--r-- 1 avidu 197609 9630 Jul  4 06:58 served_gate_a.py
-rw-r--r-- 1 avidu 197609 3754 Jun 29 13:47 splits.py
-rw-r--r-- 1 avidu 197609 15260 Jun 29 13:47 tolerance_metrics.py
-rw-r--r-- 1 avidu 197609 3782 Jun 29 13:47 training.py
-rw-r--r-- 1 avidu 197609 12623 Jun 29 20:33 windowing.py
```

==== docs =====

total 1216

```
drwxr-xr-x 1 avidu 197609  0 Jul  6 22:19 .
drwxr-xr-x 1 avidu 197609  0 Jul  6 23:47 ..
-rw-r--r-- 1 avidu 197609 3926 Jun 21 10:18 ABLATION_LAYER.md
-rw-r--r-- 1 avidu 197609 28776 Jul  5 21:35 ALPHA_LAUNCH_READINESS.md
-rw-r--r-- 1 avidu 197609 77890 Jul  5 15:34 ALPHA_LAUNCH_REVIEW_2026-07-04.md
-rw-r--r-- 1 avidu 197609 7273 Jul  6 22:19 ALPHA_TESTER_ONBOARDING_FAQ.md
-rw-r--r-- 1 avidu 197609 52912 Jun 21 10:18 ANALYZERS_AND_RECONCILERS.md
drwxr-xr-x 1 avidu 197609  0 Jun 25 16:24 archives
-rw-r--r-- 1 avidu 197609 20211 Jul  2 20:05 BACKLOG.md
-rw-r--r-- 1 avidu 197609 9604 Jun 25 16:24 CLOUD_GPU_DRYRUN_GUIDE.md
-rw-r--r-- 1 avidu 197609 91045 Jul  2 20:29 CODE_MAP.md
-rw-r--r-- 1 avidu 197609 17655 Jun 25 16:24 COMMERCIALIZATION.md
drwxr-xr-x 1 avidu 197609  0 Jun 25 16:24 COMPUTE_DECOUPLED_SERVING
-rw-r--r-- 1 avidu 197609 30814 Jul  5 21:35 CONTROL_PLANE_DURABLE_REBUILD.md
-rw-r--r-- 1 avidu 197609 7219 Jun 25 16:24 CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md
-rw-r--r-- 1 avidu 197609 12625 Jun 25 16:24 CREATING_REELS.md
-rw-r--r-- 1 avidu 197609 22072 Jul  2 21:19 DATA_IN_GCS.md
drwxr-xr-x 1 avidu 197609  0 Jul  4 06:58 data_pipeline
-rw-r--r-- 1 avidu 197609 13300 Jun 25 23:22 DECISION_SNAPSHOT_REGRESSION.md
-rw-r--r-- 1 avidu 197609 22627 Jul  2 14:51 DESKTOP_APP_PLAN.md
-rw-r--r-- 1 avidu 197609 14582 Jul  4 18:07 DOC_STATUS.md
-rw-r--r-- 1 avidu 197609 7036 Jun 21 10:18 EVALUATION.md
-rw-r--r-- 1 avidu 197609 9046 Jun 21 10:18 EXECUTION_POLICY.md
-rw-r--r-- 1 avidu 197609 15974 Jun 25 16:24 EXPERIMENT_HARNESS.md
-rw-r--r-- 1 avidu 197609 36098 Jun 25 16:24 GOLDEN_REGRESSION_FIXTURES.md
-rw-r--r-- 1 avidu 197609 8657 Jul  2 19:37 HOSTING_PLAN.md
-rw-r--r-- 1 avidu 197609 4324 Jun 25 16:24 HOW_RALLY_DETECTION_WORKS.md
-rw-r--r-- 1 avidu 197609 15140 Jul  2 08:13 I18N_PLAN.md
-rw-r--r-- 1 avidu 197609 10659 Jun 25 21:14 KHELSUTRA_PRODUCT_OVERVIEW.md
drwxr-xr-x 1 avidu 197609  0 Jun 11 00:35 mockups
-rw-r--r-- 1 avidu 197609 20582 Jun 25 16:24 MULTI_SIGNAL_FUSION_PLAN.md
-rw-r--r-- 1 avidu 197609 24525 Jul  5 15:50 NEXT_STEPS.md
-rw-r--r-- 1 avidu 197609 18861 Jun 29 20:33 NIGHTLY_ENGINE_REGRESSION.md
-rw-r--r-- 1 avidu 197609 15261 Jun 21 10:18 OWNED_MODEL_IMPLEMENTATION_PLAN.md
-rw-r--r-- 1 avidu 197609 34288 Jun 25 16:24 PACKAGING_PLAN.md
-rw-r--r-- 1 avidu 197609 13989 Jun 25 16:24 PER_VIDEO_WORKSPACE.md
-rw-r--r-- 1 avidu 197609 11766 Jun 21 10:18 PERSONALIZATION_PLAN.md
-rw-r--r-- 1 avidu 197609 49370 Jul  2 14:51 PLATFORM_ARCHITECTURE.md
-rw-r--r-- 1 avidu 197609 24523 Jun 21 10:18
POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
```

```

-rw-r--r-- 1 avidu 197609 3813 Jun 25 16:24 PROJECT_DOC_TEMPLATE.md
-rw-r--r-- 1 avidu 197609 16490 Jun 25 16:24 PROMINENT_COURT_DETECTION.md
-rw-r--r-- 1 avidu 197609 61955 Jul 2 08:13 QUALITY_ITERATIONS.md
-rw-r--r-- 1 avidu 197609 6457 Jun 25 16:24 R1_MILESTONE_LAUNCH_REPORT.md
-rw-r--r-- 1 avidu 197609 15067 Jun 25 16:24 R2_EVAL_METRIC_DESIGN.md
-rw-r--r-- 1 avidu 197609 6451 Jun 29 20:33 RALLY_DETECTION_CODE_REVIEW.md
-rw-r--r-- 1 avidu 197609 10265 Jul 2 08:13 RALLY_DETECTION_FIXES_PLAN.md
-rw-r--r-- 1 avidu 197609 16412 Jun 25 16:24 RALLY_DETECTION_GAP_CLOSURE.md
-rw-r--r-- 1 avidu 197609 32871 Jun 25 16:24 RALLY_DETECTION_QUALITY_REPORT.md
-rw-r--r-- 1 avidu 197609 45430 Jun 25 16:24 RALLY_QUALITY_RESEARCH.md
-rw-r--r-- 1 avidu 197609 19991 Jul 6 22:19 README.md
-rw-r--r-- 1 avidu 197609 16248 Jun 25 16:24 REAL_FOOTAGE_VALIDATION_REPORT.md
-rw-r--r-- 1 avidu 197609 8260 Jun 25 16:24 ROADMAP.md
-rw-r--r-- 1 avidu 197609 16120 Jun 25 16:24 SETUP_NEW_MACHINE.md
-rw-r--r-- 1 avidu 197609 11935 Jun 21 10:18 STORAGE_SHARING_MODEL.md
-rw-r--r-- 1 avidu 197609 5789 Jun 21 10:18 TRACKNET_WSL_SETUP.md
-rw-r--r-- 1 avidu 197609 6152 Jun 21 10:18 UI_ALPHA_EXECUTION_PLAN.md
-rw-r--r-- 1 avidu 197609 9034 Jun 25 16:24 UI_PROPOSAL.md
-rw-r--r-- 1 avidu 197609 11217 Jun 21 10:18 VIDEO_LOCALITY_MODEL.md

```

==== eval_baselines =====

total 64

```

drwxr-xr-x 1 avidu 197609 0 Jul 4 06:58 .
drwxr-xr-x 1 avidu 197609 0 Jul 6 23:47 ..
-rw-r--r-- 1 avidu 197609 4696 Jun 14 16:28 experiment_leaderboard.json
drwxr-xr-x 1 avidu 197609 0 Jun 25 16:24 fixtures
-rw-r--r-- 1 avidu 197609 1977 Jun 10 17:43 gemini_wrapper_2026-06-10.json
-rw-r--r-- 1 avidu 197609 2405 Jul 4 06:58 heuristic_lovo_n15.json
-rw-r--r-- 1 avidu 197609 689 Jun 10 15:57 heuristic_lovo_n6.json
-rw-r--r-- 1 avidu 197609 17504 Jul 4 06:58 nightly_baseline.json
-rw-r--r-- 1 avidu 197609 2034 Jun 25 16:24 reelcount_manifest.json

```

15. user (2026-07-06T20:11:43.742Z)

Found 173 files

```

docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md
docs\README.md
docs\CONTROL_PLANE_DURABLE_REBUILD.md
docs\ALPHA_LAUNCH_READINESS.md
backend\ui\assets\index-503c-FCB.js
backend\api\job_worker.py
tests\test_api_endpoints.py
backend\config\models.py
docs\NEXT_STEPS.md
docs\ALPHA_LAUNCH_REVIEW_2026-07-04.md
docs\DOC_STATUS.md
training\gen0\harness.py
tests\test_served_gate_a_regression.py
tests\test_golden_manifest.py
tests\test_fusion_golden_telemetry.py
tests\test_fusion_compare.py
tests\test_audio_features.py
tests\test_ablation.py
eval_baselines\nightly_baseline.json
eval_baselines\heuristic_lovo_n15.json
docs\data_pipeline\GOLDEN_VIDEOS.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_2026-07-04.md
backend\eval\ablation_pdf.py
backend\eval\ablation.py
backend\eval\calibrate_local.py
backend\eval\distill_local.py
backend\eval\fusion_audio.py
backend\eval\fusion_golden.py
backend\eval\golden_manifest.py
backend\eval\rally_seg_eval.py

```

backend\eval\rally_seq_proto.py
backend\eval\serve_contrast.py
backend\eval\served_gate_a.py
SESSION_HANDOFF.md
docs\archives\past_projects\AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md
.github\workflows\ci.yml
.github\workflows\nightly.yml
docs\DATA_IN_GCS.md
docs\CODE_MAP.md
docs\BACKLOG.md
docs\archives\past_projects\KHELSUTRA_GURU_HEALTH_REMEDIATION_PLAN-FOLDED-2026-07-02.md
docs\archives\past_projects\CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md
docs\PLATFORM_ARCHITECTURE.md
CLAUDE.md
deploy\gcp\nightly_regression\register_scheduler.sh
docs\data_pipeline\VIDEO_RECORDING_GUIDELINES.md
docs\QUALITY_ITERATIONS.md
docs\RALLY_DETECTION_FIXES_PLAN.md
deploy\gcp\buckets.env
deploy\gcp\nightly_regression\create_nightly_bucket.sh
deploy\gcp\nightly_regression\README.md
deploy\gcp\README.md
backend\pipeline\detectors\tracknet_runner.py
README.md
tests\test_windowing.py
tests\test_rally_seg_eval.py
backend\eval\windowing.py
docs\NIGHTLY_ENGINE_REGRESSION.md
docs\RALLY_DETECTION_CODE_REVIEW.md
tests\test_variant_tuning.py
tests\test_stub_runner.py
tests\test_serve_contrast.py
tests\test_rally_seg_eval_report.py
tests\test_reporting_spec_faq.py
tests\test_per_user_regression.py
tests\test_nightly_regression.py
tests\test_per_user_eval.py
tests\test_gt_loader.py
tests\test_golden_real_fixtures.py
tests\test_experiment_honest.py
tests\test_experiment.py
tests\test_export_wasb_segments.py
tests\test_golden_fixtures_split.py
tests\test_golden_fixtures.py
tests\test_calibrate_wasb.py
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\lovo_resweep.py
run_regression.py
scripts\nightly_reelcount.py
scripts\nightly_regression.py
backend\utils\run_telemetry.py
backend\tools\export.py
backend\tools\rally_slicer.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\detectors\stub_runner.py
backend\pipeline\detectors\audio_features.py
backend\eval\per_user_regression.py
backend\eval\regression.py
backend\eval\tolerance_metrics.py
backend\flywheel.py
backend\eval\per_user_eval.py
backend\eval\audio\e1_probe.py
backend\eval\calibrate_wasb.py
backend\eval\eval_partial_labels.py
backend\eval\eval_stats.py

backend\eval\experiment.py
backend\eval\export_wasb_segments.py
backend\eval\fusion_compare.py
backend\eval\golden_fixtures.py
backend\eval\golden_real_fixtures.py
backend\eval\gt_loader.py
backend\eval\annotations.py
backend\annotations__init__.py
backend\annotations\automated_provider.py
backend\annotations\base.py
backend\annotations\file_provider.py
docs\DECISION_SNAPSHOT_REGRESSION.md
scripts\register_nightly_task.ps1
scripts\nightly_regression.ps1
eval_baselines\reelcount_manifest.json
eval_baselines\fixtures\README.md
docs\data_pipeline\README.md
docs\data_pipeline\GOLDEN_SET_IMPLEMENTATION.md
docs\data_pipeline\GOLDEN_SET_PHASE1_VIDEOS.md
docs\data_pipeline\GOLDEN_DATA_SHARING.md
docs\archives\roora-vendor\ROORA_LABELING_GUIDELINES.md
docs\archives\roora-vendor\GOLDEN_SET_VENDORING.md
docs\archives\roora-vendor\README.md
docs\archives\quality-iterations\README.md
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\README.md
docs\archives\past_projects\DECOUPLED_COMPUTE\README.md
docs\archives\past_projects\DECOUPLED_COMPUTE\06-RISKS-AND-OPEN-QUESTIONS.md
docs\archives\past_projects\DECOUPLED_COMPUTE\07-COMPUTE-ABSTRACTION.md
docs\archives\past_projects\DECOUPLED_COMPUTE\04-BUILD-AND-TEST-PLAN.md
docs\archives\past_projects\DECOUPLED_COMPUTE\01-EXISTING-PLANS-AND-GAP.md
docs\archives\decisions\DECISIONS.md
docs\UI_PROPOSAL.md
docs\SETUP_NEW_MACHINE.md
docs\ROADMAP.md
docs\REAL_FOOTAGE_VALIDATION_REPORT.md
docs\RALLY_QUALITY_RESEARCH.md
docs\R1_MILESTONE_LAUNCH_REPORT.md
docs\R2_EVAL_METRIC_DESIGN.md
docs\RALLY_DETECTION_GAP_CLOSURE.md
docs\RALLY_DETECTION_QUALITY_REPORT.md
docs\PROJECT_DOC_TEMPLATE.md
docs\PROMINENT_COURT_DETECTION.md
docs\PACKAGING_PLAN.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\HOW_RALLY_DETECTION_WORKS.md
docs\EXPERIMENT_HARNESS.md
docs\GOLDEN_REGRESSION_FIXTURES.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\RUNLOG_truly-local_TEMPLATE.md
docs\COMPUTE_DECOUPLED_SERVING\TESTING_STRATEGY.md
docs\COMPUTE_DECOUPLED_SERVING\README.md
docs\COMMERCIALIZATION.md
deploy\gcp\nightly_regression\run_on_vm.sh
.gitattributes
docs\STORAGE_SHARING_MODEL.md
docs\PERSONALIZATION_PLAN.md
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
docs\EVALUATION.md
docs\ABLATION_LAYER.md
docs\ANALYZERS_AND_RECONCILERS.md
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md
docs\archives\HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
eval_baselines\gemini_wrapper_2026-06-10.json

```
eval_baselines\heuristic_lovo_n6.json
training\README.md
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md
docs\archives\quality-iterations\2026-06-initial\README.md
docs\archives\checkpoints\2026-06-strategy-foundation\README.md
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md
docs\archives\checkpoints\2026-06-scaffolding-complete\README.md
docs\archives\research\AUDIO_E1_FINDINGS.md
docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md
docs\archives\research\RALLY_DETECTION_RESEARCH.md
docs\archives\checkpoints\2026-06-eval-baseline\README.md
docs\archives\COMMERCIAL_READINESS_REVIEW.md
```

16. user (2026-07-06T20:11:43.749Z)

Found 33 files

```
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md
docs\CONTROL_PLANE_DURABLE_REBUILD.md
docs\ALPHA_LAUNCH_READINESS.md
backend\config\models.py
docs\NEXT_STEPS.md
docs\ALPHA_LAUNCH_REVIEW_2026-07-04.md
docs\DOC_STATUS.md
eval_baselines\nightly_baseline.json
backend\pipeline\detectors\native_wasb_runner.py
backend\eval\rally_seg_eval.py
SESSION_HANDOFF.md
docs\CODE_MAP.md
docs\QUALITY_ITERATIONS.md
docs\RALLY_DETECTION_FIXES_PLAN.md
tests\test_windowing.py
tests\test_rally_seg_eval.py
tests\test_tolerance_metrics.py
tests\test_rally_seg_eval_report.py
tests\test_nightly_reelcount.py
tests\test_nightly_regression.py
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\lovo_resweep.py
scripts\nightly_reelcount.py
scripts\nightly_regression.py
backend\eval\tolerance_metrics.py
eval_baselines\reelcount_manifest.json
docs\archives\quality-iterations\README.md
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\README.md
docs\archives\past_projects\DECOUPLED_COMPUTE\04-BUILD-AND-TEST-PLAN.md
docs\REAL_FOOTAGE_VALIDATION_REPORT.md
docs\R1_MILESTONE_LAUNCH_REPORT.md
docs\R2_EVAL_METRIC_DESIGN.md
docs\RALLY_DETECTION_GAP_CLOSURE.md
docs\RALLY_DETECTION_QUALITY_REPORT.md
```

17. assistant (2026-07-06T20:11:50.978Z)

The key file for shuttle-position F1/tolF1 is almost certainly `backend/eval/tolerance\_metrics.py` and `backend/eval/calibrate\_wasb.py`. The `run\_eval.py`/`run\_regression.py` harness scores rally-level IoU, not per-pixel shuttle. Let me read the tolerance metrics, calibrate_wasb, and the two key docs. Let me also check the WASB model config for the input resolution / normalization.

18. user (2026-07-06T20:11:54.938Z)

```
1      """R2 ? task-faithful rally-boundary eval (tolerance-match + over-segmentation guard).
2
3      The headline rally metric (tIoU-F1 is kept as a SECONDARY diagnostic): a prediction
```



```

55         )
56     )
57     pairs.sort()
58     used_p: set = set()
59     used_g: set = set()
60     matches: List[Match] = []
61     for _, _, i, j in pairs:
62         if i in used_p or j in used_g:
63             continue
64             used_p.add(i)
65             used_g.add(j)
66             matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
67     matches.sort(key=lambda mm: mm[1])
68     return matches
69
70
71 def _hungarian_match_1to1(
72     preds: List[Interval], gts: List[Interval], tau_start: float, tau_end: float
73 ) -> Optional[List[Match]]:
74     """Exact max-cardinality (then min total boundary error) via scipy. Returns ``None``
when
75     numpy/scipy are unavailable so the caller falls back to greedy."""
76     try:
77         import numpy as np
78         from scipy.optimize import linear_sum_assignment
79     except Exception:
80         return None
81     n, m = len(preds), len(gts)
82     size = max(n, m)
83     big = 1.0e6
84     cost = np.full((size, size), big, dtype=float)
85     for i, p in enumerate(preds):
86         for j, g in enumerate(gts):
87             if _eligible(p, g, tau_start, tau_end):
88                 cost[i, j] = abs(p[0] - g[0]) + abs(p[1] - g[1])
89     rows, cols = linear_sum_assignment(cost)
90     matches: List[Match] = []
91     for i, j in zip(rows.tolist(), cols.tolist()):
92         if i < n and j < m and cost[i, j] < big: # drop forced ineligible assignments
93             matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
94     matches.sort(key=lambda mm: mm[1])
95     return matches
96
97
98 def match_1to1(
99     preds: List[Interval],
100     gts: List[Interval],
101     tau_start: float = TAU_START,
102     tau_end: float = TAU_END,
103 ) -> Tuple[List[Match], float, float, float]:
104     """Strict 1:1 tolerance match ? ``(matches, precision, recall, f1)``. ``P = matched
/ n_pred``
105     (over-production costs precision), ``R = matched / n_gt``. Exact Hungarian when
scipy is
106     present; deterministic greedy otherwise."""
107     n, m = len(preds), len(gts)
108     if n == 0 or m == 0:
109         return [], 0.0, 0.0, 0.0
110     matches = _hungarian_match_1to1(preds, gts, tau_start, tau_end)
111     if matches is None:
112         matches = _greedy_match_1to1(preds, gts, tau_start, tau_end)
113     k = len(matches)
114     precision = k / n
115     recall = k / m
116     f1 = (

```

```

117         (2 * precision * recall / (precision + recall)) if (precision + recall) else 0.0
118     )
119     return matches, precision, recall, f1
120
121
122     def _percentile(xs: List[float], p: float) -> float:
123         """Linear-interpolated percentile (pure stdlib)."""
124         if not xs:
125             return 0.0
126         s = sorted(xs)
127         k = (len(s) - 1) * p / 100.0
128         lo = int(k)
129         hi = min(lo + 1, len(s) - 1)
130         return s[lo] + (s[hi] - s[lo]) * (k - lo)
131
132
133     def _overlap(a: Interval, b: Interval) -> float:
134         return max(0.0, min(a[1], b[1]) - max(a[0], b[0]))
135
136
137     def boundary_error_report(
138         preds: List[Interval], gts: List[Interval]
139     ) -> Dict[str, Dict[str, float]]:
140         """Median + IQR of signed AND absolute ?start, ?end, split start/end ? the "start
141         solved,
142         end open" diagnostic that aims R4.
143         Computed over **best-overlap** matches (each GT ? its maximally-overlapping
144         prediction, any
145         positive overlap), NOT the within-? 1:1 matches: a ?-gated set is survivorship-
146         biased (loose
147         ends fail the gate and vanish from the distribution), which would HIDE the very end-
148         deficit
149         this diagnostic exists to surface. Unconditional best-overlap is what the day-1
150         validation used
151         to find |?end| ? |?start|. (The within-? deficit shows up instead as low
152         ``tol_recall``.)"""
153
154     def dist(vals: List[float]) -> Dict[str, float]:
155         av = [abs(v) for v in vals]
156         return {
157             "signed_median": statistics.median(vals) if vals else 0.0,
158             "abs_median": statistics.median(av) if av else 0.0,
159             "abs_p25": _percentile(av, 25.0),
160             "abs_p75": _percentile(av, 75.0),
161             "n": float(len(vals)),
162         }
163
164     ds: List[float] = []
165     de: List[float] = []
166     for g in gts:
167         best = max(preds, key=lambda p: _overlap(p, g), default=None)
168         if best is not None and _overlap(best, g) > 0.0:
169             ds.append(best[0] - g[0])
170             de.append(best[1] - g[1])
171     return {"dstart": dist(ds), "dend": dist(de)}
172
173
174     def over_seg_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, int]:
175         """The over-segmentation GUARD (per-video no-regression in promotion):
176         ``split_count`` (?2
177         preds covering one rally) + ``merge_count`` (one pred ? ?2 rallies). Reuses the
178         bipartite
179         overlap-graph in ``segmentation_metrics.merge_split_report``."""
180         from backend.eval.segmentation_metrics import merge_split_report

```

```

174
175     r = merge_split_report(preds, gts)
176     return {"split_count": int(r["splits"]), "merge_count": int(r["merges"])}
177
178
179     # ----- #
180     # R1 ? net over-segmentation promotion guard
181     # ----- #
182     #: Default over-seg cost weights. A MERGE hides a rally (?2 golden rallies collapse into
one
183     #: predicted window ? the user can't reach the hidden one ? recall loss at rally
granularity); a
184     #: SPLIT only fragments one rally (it is still found, just in pieces). So a merge is the
strictly
185     #: worse fault and is weighted higher. (Equal weights also pass the milestone ? see the
R1 evidence;
186     #: the asymmetry is the principled default, not a number tuned to force a verdict.)
187     WEIGHT_MERGE: float = 2.0
188     WEIGHT_SPLIT: float = 1.0
189
190
191     def over_seg_guard(
192         base: List[Dict[str, Any]],
193         cand: List[Dict[str, Any]],
194         *,
195         weight_merge: float = WEIGHT_MERGE,
196         weight_split: float = WEIGHT_SPLIT,
197         use_rates: bool = True,
198         eps: float = 1e-9,
199     ) -> Dict[str, Any]:
200         """**Net** over-segmentation promotion guard (the R1 refinement of the strict per-
video rule).
201
202         ``base``/``cand`` are aligned per-video over-seg dicts (the rows ``rally_seg_eval``
already
203         emits): each needs ``split_count``/``merge_count`` and ? when ``use_rates``
(default) ?
204         ``merge_rate``/``split_rate``. Optional ``name`` keys align the two lists by video
(else by
205         position). Returns the promote-or-block verdict for promoting ``cand`` over
``base``.
206
207         **Why the strict rule needed refining.** R2 §2.3.3's guard blocks promotion if
``cand`` raises
208         ``split_count`` OR ``merge_count`` on ANY video. That is right for an *incremental*
cue (same
209         windowing structure, an increase = a real regression) but mis-fires on a
*structural* change
210         (mega-windows ? bounded): splitting a single mega-window necessarily lifts
``split_count`` from
211         ~0, and the raw merge **count** is non-monotonic ? one mega-window swallowing 40
rallies is
212         ``merge_count=1`` yet ``merge_rate=1.0`` (every rally hidden), whereas bounding it
into pieces
213         that each glue 2 neighbours is ``merge_count=9`` yet ``merge_rate=0.5`` (FEWER
rallies hidden).
214         So the strict count rule blocks a config that strictly *reduces* the fraction of
rallies lost to
215         over-merge. (Measured: the cap6+R4 milestone trips strict on 10/11 videos yet cuts
mean
216         ``merge_rate`` 0.694 ? 0.150 with NO per-video merge_rate regression ?
RALLY_QUALITY_RESEARCH §6.)
217
218         **The net rule (default verdict ``passes``):** score over-seg as ONE weighted cost
per video and

```

```

219         require BOTH:
220             1. the corpus-mean net cost does not rise (`cand_net ? base_net + eps`), AND
221             2. NO video's **merge_rate** rises beyond `eps` ? reintroducing a mega-window
that hides MORE
222             of a video's rallies is the one over-merge regression that is never acceptable,
the honest
223             "no NEW merges" rule expressed on the *rate* (the recall-meaningful quantity),
not the count.
224             With `use_rates` the per-video cost uses `merge_rate`/`split_rate` (fraction
of GT rallies
225             affected, ? [0,1] ? comparable across a structural change); `use_rates=False`
scores raw counts.
226
227             **Revert (one line):** read `strict_passes` instead of `passes` to fall back to
the original
228             strict per-video count rule. Both verdicts are always reported, so the choice is
auditable.
229             ""
230
231         def _aligned() -> List[Tuple[Dict[str, Any], Dict[str, Any]]]:
232             if (
233                 base
234                 and cand
235                 and all("name" in b for b in base)
236                 and all("name" in c for c in cand)
237             ):
238                 cmap = {c["name"]: c for c in cand}
239                 return [(b, cmap[b["name"]]) for b in base if b["name"] in cmap]
240             n = min(len(base), len(cand))
241             return [(base[i], cand[i]) for i in range(n)]
242
243         def _cost(row: Dict[str, Any]) -> float:
244             if use_rates:
245                 return weight_merge * float(row["merge_rate"]) + weight_split * float(
246                     row["split_rate"]
247                 )
248             return weight_merge * float(row["merge_count"]) + weight_split * float(
249                 row["split_count"]
250             )
251
252         pairs = _aligned()
253         n = len(pairs)
254         base_costs = [_cost(b) for b, _ in pairs]
255         cand_costs = [_cost(c) for _, c in pairs]
256         base_net = (sum(base_costs) / n) if n else 0.0
257         cand_net = (sum(cand_costs) / n) if n else 0.0
258         net_delta = cand_net - base_net
259
260         # Per-video regressions, reported for transparency.
261         new_merges: List[Dict[str, Any]] = [] # raw COUNT increases (what strict trips on)
262         new_splits: List[Dict[str, Any]] = [] # raw COUNT increases (what strict trips on)
263         merge_rate_regress: List[
264             Dict[str, Any]
265         ] = [] # RATE increases (the real over-merge regression)
266         for b, c in pairs:
267             if float(c["merge_count"]) > float(b["merge_count"]) + eps:
268                 new_merges.append(
269                     {
270                         "name": b.get("name", ""),
271                         "base": float(b["merge_count"]),
272                         "cand": float(c["merge_count"]),
273                     }
274                 )
275             if float(c["split_count"]) > float(b["split_count"]) + eps:
276                 new_splits.append(

```

```

277         {
278             "name": b.get("name", ""),
279             "base": float(b["split_count"]),
280             "cand": float(c["split_count"]),
281         }
282     )
283     if use_rates and float(c["merge_rate"]) > float(b["merge_rate"]) + eps:
284         merge_rate_regress.append(
285             {
286                 "name": b.get("name", ""),
287                 "base": float(b["merge_rate"]),
288                 "cand": float(c["merge_rate"]),
289             }
290         )
291
292     strict_passes = (not new_merges) and (not new_splits)
293     net_cost_ok = net_delta <= eps
294     no_merge_rate_regress = not merge_rate_regress
295     passes = bool(net_cost_ok and no_merge_rate_regress)
296     return {
297         "passes": passes,
298         "strict_passes": strict_passes,
299         "net_cost_ok": net_cost_ok,
300         "no_merge_rate_regress": no_merge_rate_regress,
301         "base_net": base_net,
302         "cand_net": cand_net,
303         "net_delta": net_delta,
304         "n": n,
305         "use_rates": use_rates,
306         "weight_merge": weight_merge,
307         "weight_split": weight_split,
308         "new_merges": new_merges,
309         "new_splits": new_splits,
310         "merge_rate_regress": merge_rate_regress,
311     }
312
313
314     def tolerance_report(
315         preds: List[Interval],
316         gts: List[Interval],
317         tau_start: float = TAU_START,
318         tau_end: float = TAU_END,
319     ) -> Dict[str, object]:
320         """The full R2 block for one video: P/R/F1@? + boundary-error distributions + over-
321         seg guard."""
322         matches, precision, recall, f1 = match_1to1(preds, gts, tau_start, tau_end)
323         return {
324             "tau_start": tau_start,
325             "tau_end": tau_end,
326             "n_pred": len(preds),
327             "n_gt": len(gts),
328             "matched": len(matches),
329             "precision": precision,
330             "recall": recall,
331             "f1": f1,
332             "boundary_error": boundary_error_report(preds, gts),
333             "over_seg": over_seg_report(preds, gts),
334         }
335
336     def tolerance_sweep(
337         preds: List[Interval], gts: List[Interval], taus: Tuple[float, ...] = SWEEP_TAUS
338     ) -> Dict[float, float]:
339         """Symmetric-? F1 sweep ? the trend curve (a single ? misleads). Monotonic non-
340         decreasing in ?."""

```

```
340         return {t: match_1to1(preds, gts, t, t)[3] for t in taus}
341
```

19. user (2026-07-06T20:11:54.968Z)

```
1     """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
2     "+X%".
3
4     Wires the already-tested pieces together:
5     WASB trajectory CSV (Frame,Visibility,X,Y)
6     -> TrackNetRunner.trajectory_to_action_windows(cfg)    [predict_fn]
7     -> backend.eval.calibration (improvement_report / cross_validate)
8
9     Run (after a full-video WASB pass exists):
10    python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
11    --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
12
13    Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
14    same GT is an **optimistic** upper bound. The trustworthy number is the
15    cross-validated held-out **recall** (and, once a 2nd labelled video exists,
16    `leave_one_video_out` for precision/F1 generalization).
17    """
18
19    from __future__ import annotations
20
21    import argparse
22    from typing import Callable, List, Tuple
23
24    from backend.eval.calibration import (
25        cross_validate,
26        evaluate,
27        improvement_report,
28        select_best,
29    )
30
31    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
32
33    Interval = Tuple[float, float]
34    Config = Tuple[float, float, float] # (velocity_thresh, merge_gap, min_window_duration)
35
36    BASELINE: Config = (
37        0.02,
38        2.0,
39        2.0,
40    ) # the WasbConfig / trajectory_to_action_windows defaults
41
42    def load_golden_gts(csv_path: str) -> List[Interval]:
43        """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
44
45        Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
46        start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
47        driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
48        """
49
50        from backend.eval.gt_loader import load_gt_intervals
51
52        return load_gt_intervals(csv_path, strict=False)
53
54    def make_predict_fn(
55        trajectory_csv: str, fps: float, frame_width: float
56    ) -> Callable[[Config], List[Interval]]:
57        """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
58        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
59
60        def predict_fn(cfg: Config) -> List[Interval]:
```

```

60         vt, mg, mwd = cfg
61         wins = TrackNetRunner.trajectory_to_action_windows(
62             points,
63             fps=fps,
64             frame_width=frame_width,
65             velocity_thresh=vt,
66             merge_gap=mg,
67             min_window_duration=mwd,
68         )
69         return [(w["start"], w["end"]) for w in wins]
70
71     return predict_fn
72
73
74 def default_grid() -> List[Config]:
75     return [
76         (vt, mg, mwd)
77         for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
78         for mg in (1.0, 2.0, 3.0)
79         for mwd in (1.0, 2.0, 3.0)
80     ]
81
82
83 def run(
84     trajectory_csv: str,
85     labels_csv: str,
86     fps: float,
87     frame_width: float,
88     iou: float = 0.5,
89 ) -> dict:
90     gts = load_golden_gts(labels_csv)
91     if not gts:
92         raise SystemExit(f"no usable golden rallies in {labels_csv}")
93     predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
94     grid = default_grid()
95
96     base = evaluate(predict_fn(BASELINE), gts, iou)
97     best_cfg, best_f1 = select_best(
98         predict_fn, grid, gts, metric="f1", iou_threshold=iou
99     )
100     fit = improvement_report(
101         predict_fn, BASELINE, best_cfg, gts, iou
102     ) # OPTIMISTIC (fit on full GT)
103     cv = cross_validate(predict_fn, grid, gts, k=5) # HONEST held-out recall
104
105     print("=== WASB rally-segmentation calibration ===")
106     print(f"golden rallies: {len(gts)} | IoU thresh: {iou}")
107     print(
108         f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
109         f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}"
110     )
111     print(f"tuned cfg {best_cfg}: f1={best_f1:.3f}")
112     print(
113         f" precision {fit['baseline']['precision']:.3f} ->
{fit['tuned']['precision']:.3f} "
114         f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)"
115     )
116     print(
117         f" f1 {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
118         f"({fit['f1_delta_pct']:+.1f}%)"
119     )
120     print(
121         f"HONEST (cross-val, {cv.get('folds', '?')} folds): held-out recall "
122         f"{cv.get('mean_test', float('nan')):.3f} ± {cv.get('std_test', 0):.3f}, "

```

```

123         f"generalization_gap {cv.get('generalization_gap', float('nan')):+.3f}"
124     )
125     print(
126         "(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)"
127     )
128     return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}
129
130
131 def main():
132     ap = argparse.ArgumentParser()
133     ap.add_argument(
134         "--trajectory", required=True, help="WASB trajectory CSV (Frame,Visibility,X,Y)"
135     )
136     ap.add_argument(
137         "--labels", required=True, help="golden labels CSV (start_time/end_time cols)"
138     )
139     ap.add_argument("--fps", type=float, default=59.94)
140     ap.add_argument("--frame-width", type=float, default=1920.0)
141     ap.add_argument("--iou", type=float, default=0.5)
142     args = ap.parse_args()
143     run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
144
145
146 if __name__ == "__main__":
147     main()
148

```

20. user (2026-07-06T20:11:54.985Z)

```

1     """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3     The repo historically carried two incompatible loaders:
4     - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5       ``curate export`` format), raises on bad rows;
6     - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7       (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9     A file in one convention fed to the other loader yielded zero/garbage intervals with no
10    error. This module accepts BOTH conventions (header-mapped, else positional), so every
11    quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12    flows through one reader. Both legacy entry points now delegate here.
13
14    Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15    - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16      columns ? rally_number, ending_reason, sport, ? ? are ignored);
17    - no header: columns 0,1 positionally.
18    Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19    (``annotations.parse_timestamp``).
20    """
21
22    import csv
23    import logging
24    from typing import List, Optional, Tuple
25
26    from backend.eval.annotations import parse_timestamp
27
28    logger = logging.getLogger(__name__)
29
30    Interval = Tuple[float, float]
31
32    _START_NAMES = ("start", "start_time")
33    _END_NAMES = ("end", "end_time")

```

```

34
35 def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
36     """If `row` is a header naming start/end columns, return their indices, else
None."""
37     cells = [(c or "").strip().lower() for c in row]
38     start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
39     end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
40     if start_idx is not None and end_idx is not None:
41         return start_idx, end_idx
42     return None
43
44
45 def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
46     """Load GT rally intervals from either golden-CSV convention.
47
48     strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
49     (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
50     strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
51     skipped ? a silently shrinking GT set flatters precision.
52
53     Returns intervals sorted by start; overlapping GT intervals are logged as a warning
54     (they break one-to-one matching and usually indicate a labeling error).
55     """
56     intervals: List[Interval] = []
57     skipped = 0
58     cols: Optional[Tuple[int, int]] = None # (start_idx, end_idx); None until detected
59     saw_header = False
60
61     with open(csv_path, newline="", encoding="utf-8-sig") as f:
62         for lineno, row in enumerate(csv.reader(f), start=1):
63             if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
64                 continue
65             if cols is None and not saw_header:
66                 header = _header_columns(row)
67                 if header is not None:
68                     cols = header
69                     saw_header = True
70                     continue # consume the header row
71             cols = (0, 1) # headerless: positional start,end
72             assert cols is not None # every branch above assigns it
73             si, ei = cols
74             try:
75                 if len(row) <= max(si, ei):
76                     raise ValueError(
77                         f"expected at least {max(si, ei) + 1} columns, got {row!r}"
78                     )
79                 start = parse_timestamp(row[si])
80                 end = parse_timestamp(row[ei])
81                 if start < 0:
82                     raise ValueError(f"negative start ({start})")
83                 if start >= end:
84                     raise ValueError(f"start ({start}) >= end ({end})")
85             except ValueError as e:
86                 if strict:
87                     raise ValueError(f"{csv_path}:{lineno}: {e}") from e
88                 skipped += 1
89                 continue
90             intervals.append((start, end))
91
92     if skipped:
93         logger.warning(
94             "%s: skipped %d malformed GT row(s) (strict=False)", csv_path, skipped
95         )
96
97     intervals.sort(key=lambda iv: iv[0])

```

```

98         for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
99             if s2 < e1:
100                 logger.warning(
101                     "%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
102                     "labeling error? Overlaps break one-to-one matching.",
103                     csv_path,
104                     s1,
105                     e1,
106                     s2,
107                     e2,
108                 )
109         return intervals
110

```

21. assistant (2026-07-06T20:11:59.590Z)

Now I understand the landscape clearly. The repo's F1/tolF1 machinery is **rally-boundary** oriented (start/end time intervals), NOT per-pixel shuttle-position F1. The user is asking about "shuttle detector F1/tolF1 on golden videos" which ? for a shuttle detector ? is the classic **WASB** per-frame position F1 within a ?-pixel tolerance (the TrackNet/WASB standard metric). This may live elsewhere. Let me search for per-frame shuttle position scoring and the WASB config, plus read the key docs.

22. user (2026-07-06T20:12:03.237Z)

Found 19 files

- backend\pipeline\segmenters\trajectory_hybrid.py
- backend\ui\assets\index-dvIzm83F.css
- backend\ui\assets\index-503c-FCB.js
- backend\storage\gcs.py
- backend\config\models.py
- backend\pipeline\detectors\wasb_infer.py
- backend\pipeline\detectors\native_wasb_runner.py
- backend\eval\fusion_audio.py
- backend\eval\rally_seg_eval.py
- backend\pipeline\detectors\tracknet_runner.py
- backend\eval>windowing.py
- backend\pipeline\personalization\serving.py
- backend\eval\tolerance_metrics.py
- backend\eval\batch.py
- backend\eval\eval_partial_labels.py
- backend\eval\experiment.py
- backend\eval\fusion_compare.py
- backend\eval\golden_fixtures.py
- backend\eval\harness.py

23. user (2026-07-06T20:12:05.101Z)

```

1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so no WASB re-run), needs no Gemini and no DB,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):

```

```

12         python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
13     --out-dir output
14
15     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
16     re-run
17     `fusion_compare` with different variants/thresholds forever without re-detecting on the
18     GPU.
19     """
20
21     from __future__ import annotations
22
23     import argparse
24     import json
25     import os
26     import time
27     from typing import Any, Dict, List, Optional, Sequence
28
29     from backend.eval import distill_local, experiment
30     from backend.eval.calibrate_wasb import load_golden_gts
31     from backend.eval.golden_manifest import load_resolved_manifest
32     from backend.eval.training import label_candidates
33     from backend.pipeline.detectors import fusion_features as ff
34     from backend.pipeline.detectors import rally_gate
35     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
36     from backend.utils.run_telemetry import RunTelemetry
37
38     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
39     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
40     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
41     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
42     SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
43
44     def is_full_fusion_feature_file(path: str) -> bool:
45         """True when an existing golden feature JSON already has shuttle+person cue dicts.
46
47         Corpus-growth runs can leave leaner windowing-only JSONs with the same filename
48         pattern.
49
50         Those are useful for gts/windowing analysis, but they are not the replayable fusion
51         substrate that ``fusion_compare`` and ``ablation`` need.
52         """
53
54         try:
55             with open(path, encoding="utf-8") as f:
56                 data = json.load(f)
57         except (OSError, json.JSONDecodeError, TypeError):
58             return False
59         candidates = data.get("candidates")
60         if not isinstance(candidates, list):
61             return False
62         return experiment._is_fusion_schema(candidates)
63
64     def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
65         """Load ``video_id -> local_path`` from the collector golden-source manifest.
66
67         The collector records source/proxy ownership and full-file MD5s. Only entries marked
68         ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
69         """
70
71         if not source_manifest_path:
72             return {}
73         with open(source_manifest_path, encoding="utf-8") as f:
74             data = json.load(f)
75         entries = data.get("videos", data) if isinstance(data, dict) else data

```

```

73         if not isinstance(entries, list):
74             raise ValueError(f"source manifest must contain a videos list:
{source_manifest_path}")
75
76         paths: Dict[str, str] = {}
77         for entry in entries:
78             if not isinstance(entry, dict):
79                 continue
80             video_id = entry.get("video_id") or entry.get("name")
81             local_path = entry.get("local_path")
82             if not video_id or not local_path:
83                 continue
84             if entry.get("source_status", "present_local") != "present_local":
85                 continue
86             paths[str(video_id)] = str(local_path)
87         return paths
88
89
90     def attach_source_video_paths(
91         specs: Sequence[Dict[str, Any]], source_manifest_path: Optional[str]
92     ) -> int:
93         """Attach explicit ``video_path`` fields to manifest specs from the source manifest.
94
95         Returns the number of specs updated. The collector source manifest is authoritative
96         for local source/proxy video location and overrides derived/stale manifest paths.
97         """
98
99         source_paths = load_source_video_paths(source_manifest_path)
100         updated = 0
101         for spec in specs:
102             name = str(spec.get("name", ""))
103             source_path = source_paths.get(name)
104             if not source_path:
105                 continue
106             if spec.get("video_path") == source_path:
107                 continue
108             spec["video_path"] = source_path
109             updated += 1
110         return updated
111
112
113     def _filter_specs(
114         specs: Sequence[Dict[str, Any]], only: Sequence[str]
115     ) -> List[Dict[str, Any]]:
116         if not only:
117             return list(specs)
118         wanted = list(dict.fromkeys(only))
119         wanted_set = set(wanted)
120         filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
121         found = {str(spec.get("name")) for spec in filtered}
122         missing = [name for name in wanted if name not in found]
123         if missing:
124             raise ValueError(f"--only requested unknown video(s): {' '.join(missing)}")
125         return filtered
126
127
128     def _derive_video_path(spec: Dict[str, Any]) -> str:
129         """Original MP4 for a manifest entry ? explicit ``video_path``, else derived from the
130         golden labels path (``X.rallies.csv`` -> ``X.MP4``, the golden-set naming
131         convention)."""
132         if spec.get("video_path"):
133             return str(spec["video_path"])
134         labels = str(spec["labels"])
135         if labels.endswith(".rallies.csv"):
136             return labels[: -len(".rallies.csv")] + ".MP4"

```

```

136         raise ValueError(
137             f"{spec.get('name')}: no video_path and labels {labels!r} isn't *.rallies.csv"
138         )
139
140
141     def _raise_for_missing_source_videos(
142         specs: Sequence[Dict[str, Any]], *, context: str
143     ) -> None:
144         missing: List[str] = []
145         for spec in specs:
146             name = str(spec.get("name", "<unnamed>"))
147             try:
148                 video = _derive_video_path(spec)
149             except ValueError as e:
150                 missing.append(f"{name}: {e}")
151                 continue
152             if not os.path.isfile(video):
153                 missing.append(f"{name}: {video}")
154         if missing:
155             raise FileNotFoundError(
156                 f"source video(s) missing for {context}: {'', '.join(missing)}"
157             )
158
159
160     def extract_video(
161         spec: Dict[str, Any],
162         detector: Optional[Any] = None,
163         frame_skip: int = 3,
164         iou: float = 0.5,
165         log_every_sec: float = 15.0,
166         telemetry: Optional[Any] = None,
167         use_gpu: bool = True,
168         detector_model: Optional[str] = None,
169         max_gpu_temp_c: Optional[float] = None,
170     ) -> Dict[str, Any]:
171         """One golden video -> a feature record (shuttle + person features per candidate
172         window).
173
174         Returns ``{name, fps, frame_width,
175         candidates:[{start,end,shuttle{},person{},label}], gts}``.
176         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
177         tests;
178         when None a real `PersonDetector` is built (loads torchvision on first use).
179         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
180         snapshots
181         from the detection progress callback so a hard kill leaves a forensic runlog.
182         """
183         fps, fw = float(spec["fps"]), float(spec["frame_width"])
184         traj = str(spec["trajectory"])
185         points = TrackNetRunner.parse_trajectory_csv(traj)
186         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
187         gts = load_golden_gts(str(spec["labels"]))
188         labels = label_candidates(intervals, gts, iou)
189         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
190         estimate
191         # (parity with the shuttle net-crossing gate) rather than hard-coding an
192         orientation.
193         axis, net_px, _span = rally_gate.estimate_play_axis(points)
194         video_path = _derive_video_path(spec)
195         if not os.path.isfile(video_path):
196             raise FileNotFoundError(
197                 f"{spec.get('name')}: source video not found for fusion extraction:
198                 {video_path}"
199             )
200

```

```

194     if detector is None:
195         from backend.pipeline.detectors.person_detector import PersonDetector
196
197         indexing_cfg: Dict[str, Any] = {"use_gpu": use_gpu}
198         if detector_model:
199             indexing_cfg["detector_model"] = detector_model
200         detector = PersonDetector({"indexing": indexing_cfg})
201
202     # Single-pass person detection over ALL candidate windows (open the video ONCE, read
203     # forward ? no per-window seek). Then compute features per window (pure, no GPU).
204     n = len(intervals)
205     win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
206     print(
207         f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...",
208         flush=True,
209     )
210     t0 = time.monotonic()
211     state = {"last": t0}
212
213     def _progress(done: int, total: int) -> None:
214         rec = None
215         if telemetry is not None:
216             rec = telemetry.snapshot(done, total, phase=spec["name"])
217         if max_gpu_temp_c is not None and rec is not None:
218             gpu = rec.get("gpu") or {}
219             temp = gpu.get("temp_c")
220             if temp is not None and float(temp) >= max_gpu_temp_c:
221                 raise RuntimeError(
222                     f"{spec['name']}: GPU temperature {float(temp):.1f}C reached "
223                     f"--max-gpu-temp-c {max_gpu_temp_c:.1f}; aborting before thermal
shutdown"
224                 )
225         now = time.monotonic()
226         if now - state["last"] >= log_every_sec or done >= total:
227             el = now - t0
228             rate = done / el if el > 0 else 0.0
229             eta = (total - done) / rate if rate > 0 else 0.0
230             pct = 100 * done // total if total else 0
231             print(
232                 f"[fusion_golden]   {spec['name']}: detect frame {done}/{total} ({pct}%)
"
233                 f"| {el:.0f}s elapsed | ETA {eta:.0f}s",
234                 flush=True,
235             )
236             state["last"] = now
237
238     per_window = detector.detections_over_windows(
239         video_path, win_frames, frame_skip, progress_cb=_progress
240     )
241
242     candidates: List[Dict[str, Any]] = []
243     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
244         det = per_window[i]
245         det_fw = det.get("frame_width") or fw
246         det_fh = det.get("frame_height") or 0.0
247         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
248         # normalisation fusion_features uses (x/width, y/height).
249         if axis == "x":
250             net_norm = net_px / det_fw if det_fw > 0 else 0.5
251         else:
252             net_norm = net_px / det_fh if det_fh > 0 else 0.5
253         sf, ef = win_frames[i]
254         shuttle_in = [p for p in points if sf <= p.frame <= ef]

```

```

255         person = ff.compute_person_features(
256             det["frames"],
257             shuttle_in,
258             det_fw,
259             det_fh,
260             court_polygon=None,
261             net=(axis, net_norm),
262         )
263     candidates.append(
264         {
265             "start": float(s),
266             "end": float(e),
267             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
268             "person": {k: float(v) for k, v in person.items()},
269             "label": int(labels[i]),
270         }
271     )
272     return {
273         "name": spec["name"],
274         "fps": fps,
275         "frame_width": fw,
276         "candidates": candidates,
277         "gts": [[float(a), float(b)] for a, b in gts],
278     }
279
280
281 def run(
282     manifest_path: str,
283     out_dir: str,
284     frame_skip: int = 3,
285     iou: float = 0.5,
286     fresh: bool = False,
287     smallest_first: bool = False,
288     source_manifest: Optional[str] = None,
289     only: Sequence[str] = (),
290     missing_fusion_only: bool = False,
291     use_gpu: bool = True,
292     detector_model: Optional[str] = None,
293     max_gpu_temp_c: Optional[float] = None,
294 ) -> List[str]:
295     specs = load_resolved_manifest(manifest_path)
296     source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
297     if source_manifest:
298         updated = attach_source_video_paths(specs, source_manifest)
299         print(
300             f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
301             f"from {source_manifest}",
302             flush=True,
303         )
304     specs = _filter_specs(specs, only)
305     if smallest_first:
306         # Process small videos first for quick feedback (the slow full match lands
307         last);
308         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
309         row/frame).
310         specs = sorted(
311             specs,
312             key=lambda s: (
313                 os.path.getsize(s["trajectory"])
314                 if os.path.isfile(s.get("trajectory", ""))
315                 else 0
316             ),
317         )
318     os.makedirs(out_dir, exist_ok=True)
319     # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so

```

```

keep a tiny
318     # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
319     # break extraction, so guard the build and fall back to no telemetry on any error.
320     telem: Optional[RunTelemetry]
321     try:
322         telem = RunTelemetry(
323             os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden"
324         )
325     except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
326         telem = None
327     written: List[str] = []
328     specs_to_extract: List[Dict[str, Any]] = []
329     for spec in specs:
330         out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
331         existing = os.path.isfile(out)
332         existing_full = existing and is_full_fusion_feature_file(out)
333         if fresh:
334             specs_to_extract.append(spec)
335         elif missing_fusion_only:
336             if existing and not existing_full:
337                 specs_to_extract.append(spec)
338             elif not existing or not existing_full:
339                 specs_to_extract.append(spec)
340         _raise_for_missing_source_videos(specs_to_extract, context="fusion extraction")
341
342     for vi, spec in enumerate(specs, 1):
343         out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
344         existing = os.path.isfile(out)
345         existing_full = existing and is_full_fusion_feature_file(out)
346         # Resumable: skip a video whose full-fusion JSON already exists. A leaner
347         # windowing-only JSON is stale for A7, so re-extract it even without --fresh.
348         if missing_fusion_only and not fresh and existing_full:
349             print(
350                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
351                 f"features already present -> {out}",
352                 flush=True,
353             )
354             written.append(out)
355             continue
356         if missing_fusion_only and not fresh and not existing:
357             print(
358                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: no existing "
359                 f"feature JSON -> skipping (--missing-fusion-only upgrades existing "
360                 f"non-fusion JSONs only)",
361                 flush=True,
362             )
363             continue
364         if not fresh and existing and existing_full:
365             print(
366                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted
367 "
368                 f"-> {out} (use --fresh to redo)",
369                 flush=True,
370             )
371             written.append(out)
372             continue
373         if not fresh and existing and not existing_full:
374             print(
375                 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
376                 f"is not full-fusion schema; re-extracting...",
377                 flush=True,
378             )
379             print(

```

```

379         f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
380         flush=True,
381     )
382     t0 = time.monotonic()
383     if telem is not None:
384         telem.event("video_start", spec["name"])
385     try:
386         data = extract_video(
387             spec,
388             frame_skip=frame_skip,
389             iou=iou,
390             telemetry=telem,
391             use_gpu=use_gpu,
392             detector_model=detector_model,
393             max_gpu_temp_c=max_gpu_temp_c,
394         )
395     except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
396         if telem is not None:
397             telem.event("error", spec["name"], err=str(e))
398         raise
399     n_pos = sum(c["label"] for c in data["candidates"])
400     with open(out, "w", encoding="utf-8") as f:
401         json.dump(data, f, indent=2)
402     if telem is not None:
403         telem.event(
404             "video_done",
405             spec["name"],
406             windows=len(data["candidates"]),
407             positives=n_pos,
408         )
409     print(
410         f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
411         f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
412         f"-> {out}",
413         flush=True,
414     )
415     written.append(out)
416     return written
417
418
419 def main() -> None:
420     ap = argparse.ArgumentParser(
421         description="Extract golden-set shuttle+person features (fusion P3 GPU step)."
422     )
423     ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
424     ap.add_argument("--out-dir", default="output")
425     ap.add_argument("--frame-skip", type=int, default=3)
426     ap.add_argument("--iou", type=float, default=0.5)
427     ap.add_argument(
428         "--fresh", action="store_true", help="re-extract even if a JSON already exists"
429     )
430     ap.add_argument(
431         "--smallest-first",
432         action="store_true",
433         help="process smaller videos first for quick feedback (LOVO is order-
independent)",
434     )
435     ap.add_argument(
436         "--source-manifest",
437         default=None,
438         help=(
439             "collector golden_source_videos.json; overrides video paths for source/proxy

```

```

"
440         f"videos (or set {SOURCE_MANIFEST_ENV})"
441     ),
442 )
443 ap.add_argument(
444     "--only",
445     action="append",
446     default=[],
447     metavar="VIDEO_ID",
448     help="process one manifest video by name; repeatable",
449 )
450 ap.add_argument(
451     "--missing-fusion-only",
452     action="store_true",
453     help=(
454         "upgrade existing feature JSONs that lack full shuttle+person fusion schema;
455         "skip absent outputs and already-full outputs"
456     ),
457 )
458 ap.add_argument(
459     "--cpu",
460     action="store_true",
461     help="run torchvision person detection on CPU instead of CUDA",
462 )
463 ap.add_argument(
464     "--detector-model",
465     default=None,
466     choices=[
467         "fasterrcnn_resnet50_fpn",
468         "fasterrcnn_mobilenet_v3_large_fpn",
469         "retinanet_resnet50_fpn",
470     ],
471     help="torchvision detector model; default keeps the configured Faster R-CNN
ResNet",
472 )
473 ap.add_argument(
474     "--max-gpu-temp-c",
475     type=float,
476     default=None,
477     help="abort extraction if telemetry sees GPU temperature at or above this
value",
478 )
479 args = ap.parse_args()
480 run(
481     args.manifest,
482     args.out_dir,
483     args.frame_skip,
484     args.iou,
485     fresh=args.fresh,
486     smallest_first=args.smallest_first,
487     source_manifest=args.source_manifest,
488     only=args.only,
489     missing_fusion_only=args.missing_fusion_only,
490     use_gpu=not args.cpu,
491     detector_model=args.detector_model,
492     max_gpu_temp_c=args.max_gpu_temp_c,
493 )
494
495
496 if __name__ == "__main__":
497     main()
498

```

Found 30 files

```
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md
backend\config\models.py
deploy\cloudrun\README.md
deploy\cloudrun\Dockerfile
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
tests\test_wasb_cache.py
tests\test_native_wasb_runner.py
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_2026-07-04.md
backend\pipeline\detectors\wasb_infer.py
docs\DATA_IN_GCS.md
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\GPU_VM_SERVING.md
docs\CODE_MAP.md
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\docs\ANNOTATE_AND_TRAIN.md
deploy\gcp\nightly_regression\launch.sh
deploy\gcp\buckets.env
deploy\gcp\nightly_regression\README.md
deploy\gcp\README.md
backend\pipeline\detectors\wasb_runner.py
tests\test_experiment.py
docs\archives\past_projects\DECOUPLED_COMPUTE\DEPLOY_REPORT_GCP_2026-06-20.md
docs\archives\past_projects\DECOUPLED_COMPUTE\04-BUILD-AND-TEST-PLAN.md
docs\SETUP_NEW_MACHINE.md
docs\PACKAGING_PLAN.md
docs\CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md
docs\CLOUD_GPU_DRYRUN_GUIDE.md
deploy\gcp\nightly_regression\run_on_vm.sh
deploy\digitalocean\gpu_setup.sh
docs\TRACKNET_WSL_SETUP.md
config.json
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md
```

25. assistant (2026-07-06T20:12:17.212Z)

Now I understand the eval landscape fully: the entire golden harness here is ****rally-boundary**** (time-interval) F1, not per-pixel shuttle F1. There is NO per-pixel shuttle-position F1/tolF1 harness in this repo ? the shuttle detector's output feeds into rally windowing which is what's scored. This is a critical finding for the user.

The actual WASB model config (inp_width, HRNet, weight loading) lives in the WASB-SBDT repo (~\models\WASB-SBDT\), which is external. Let me check if it's cloned locally, read the WASB config models, and the key docs. Let me delegate the doc-reading and config-hunting in parallel while I check the WASB repo.

26. user (2026-07-06T20:13:31.474Z)

=== WASB repo locations ===

```
--- /c/Users/avidu/models/WASB-SBDT ---
--- C:/Users/avidu/models/WASB-SBDT ---
--- /c/Users/avidu/models/WASB-SBDT ---
```

=== search for WASB configs anywhere under khelsutra-guru ===

=== any eval.yaml / wasb.yaml / badminton.yaml ===

```
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/reporting/spec.yaml
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/deploy/cloudrun/cloudbuild.yaml
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/deploy/gcp/nightly_regression/cloudbuild.launcher.yaml
```

=== pretrained weights ===

```
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/stage/wasb_badminton_best.pth.tar
```

27. user (2026-07-06T20:13:31.614Z)

```
48:     repo_dir: str = "~/models/TrackNetV4"
51:     weights_path: str = ""
77:     repo_dir: str = "~/models/WASB-SBDT"
80:     weights_path: str = (
92:     python_bin: str = (
```

28. assistant (2026-07-06T20:13:37.191Z)

The WASB-SBDT repo is NOT cloned locally (only an empty dir at C:/Users/avidu/models/WASB-SBDT). The model config (HRNet build, inp_width/height, normalization, heatmap postprocess) lives in that external repo which I can't read here. But there's a weights file staged at `scratch/stage/wasb\_badminton\_best.pth.tar` ? I can inspect its keys directly to answer the weight-loading question. Let me also check the WasbConfig in models.py and look for any WASB inference details in the export/segment code that reveal the postprocess.

Let me delegate the doc-reading to a subagent and simultaneously inspect the weights file and config.

29. user (2026-07-06T20:13:38.185Z)

```
40         )
41         relevance: ValidationRelevanceConfig = Field(
42             default_factory=ValidationRelevanceConfig
43         )
44
45
46     class TrackNetConfig(BaseModel):
47         wsl_distro: str = "Ubuntu"
48         repo_dir: str = "~/models/TrackNetV4"
49         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50         conda_env: str = "TrackNetV4"
51         weights_path: str = ""
52         queue_length: int = 5
53         stage_in_wsl: bool = True
54         wsl_stage_dir: str = "~/clips"
55         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56         velocity_threshold: float = 0.02
57         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58         # decoded frame cache) are deleted automatically ? they are pure intermediates,
59         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61         keep_frames: bool = False
62         # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
63         # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
64         # 0 = no extra floor (the headroom guard still applies).
65         min_free_gb: float = 0.0
66
67
68     class WasbIndexConfig(BaseModel):
69         """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72         these knobs actually take effect ? previously this whole block was silently dropped
73         because nested config sections defaulted to extra='ignore'.
74         """
75
76         wsl_distro: str = "Ubuntu"
77         repo_dir: str = "~/models/WASB-SBDT"
78         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79         conda_env: str = "wasb"
80         weights_path: str = (
```

```

81         "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82     )
83     sport: str = "badminton"
84     wsl_stage_dir: str = "~/clips"
85     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86     velocity_threshold: float = 0.02
87     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
88     ---
89     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
90     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
91     # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
92     device: str = "cuda" # auto | cuda | mps | cpu
93     python_bin: str = (
94         "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
95     )
96     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
97     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
98     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
99     keep_frames: bool = False
100    # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
101    # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
102    # 0 = no extra floor (the headroom guard still applies).
103    min_free_gb: float = 0.0
104    # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
105    # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
106    # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
107    # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
108    # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
109    # Tri-state:
110    # None = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
111    # behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
112    # True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
113    stream_video: Optional[bool] = None
114    # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
115    # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
116    # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
117    # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
118    # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
119    batch_size: Optional[int] = None
120    # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly

```

30. user (2026-07-06T20:13:49.816Z)

TOP-LEVEL TYPE: <class 'dict'>

TOP-LEVEL KEYS: ['model_state_dict']

model_state_dict: dict-like, 428 keys; first10=['conv1.weight', 'bn1.weight', 'bn1.bias', 'bn1.running_mean', 'bn1.running_var', 'bn1.num_batches_tracked', 'conv2.weight', 'bn2.weight', 'bn2.bias', 'bn2.running_mean']

31. user (2026-07-06T20:13:55.773Z)

Async agent launched successfully.

agentId: ac396efe913fa3264 (internal ID - do not mention to user. Use SendMessage with to: 'ac396efe913fa3264', summary: '<5-10 word recap>' to continue this agent.)

The agent is working in the background. You will be notified automatically when it completes.

Do not duplicate this agent's work ? avoid working with the same files or topics it is using.

output_file: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-

highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\tasks\ac396efe913fa3264.output

Do NOT Read or tail this file via the shell tool ? it is the full subagent JSONL transcript and reading it will overflow your context. If the user asks for progress, say the agent is still running; you'll get a completion notification.

32. assistant (2026-07-06T20:14:01.522Z)

The checkpoint has exactly one top-level key: `model\_state\_dict` (428 keys, HRNet layers starting `conv1.weight`, `bn1.\*`). That directly answers the weight-load question. Now let me get the WASB model config values. Since the WASB-SBDT repo isn't cloned, the definitive config lives upstream ? but the repo may have staged/cached copies, or the values appear in export_wasb_segments.py / other WASB-touching code. Let me check the weights further (conv1 shape reveals input channel count = frames_in × 3) and search for any cached WASB config values in the repo.

33. user (2026-07-06T20:14:11.198Z)

conv1.weight shape: (64, 9, 3, 3) -> in_channels = 9

LAST 12 keys:

```
stage4.0.fuse_layers.3.1.1.1.bias (128,)
stage4.0.fuse_layers.3.1.1.1.running_mean (128,)
stage4.0.fuse_layers.3.1.1.1.running_var (128,)
stage4.0.fuse_layers.3.1.1.1.num_batches_tracked ()
stage4.0.fuse_layers.3.2.0.0.weight (128, 64, 3, 3)
stage4.0.fuse_layers.3.2.0.1.weight (128,)
stage4.0.fuse_layers.3.2.0.1.bias (128,)
stage4.0.fuse_layers.3.2.0.1.running_mean (128,)
stage4.0.fuse_layers.3.2.0.1.running_var (128,)
stage4.0.fuse_layers.3.2.0.1.num_batches_tracked ()
final_layers.0.weight (3, 16, 1, 1)
final_layers.0.bias (3,)
HEAD? final_layers.0.weight (3, 16, 1, 1)
HEAD? final_layers.0.bias (3,)
```

34. user (2026-07-06T20:14:14.892Z)

=== staged dir contents (maybe configs live here) ===

scratch/stage:

Badminton BXH 2_proxy.rallies.csv

Boxhill Doubles_proxy.rallies.csv

GX010128.rallies.csv

GX020094_proxy.rallies.csv

GX030094_proxy.rallies.csv

mahadevpura-1.rallies.csv

mahadevpura-2_proxy.rallies.csv

wasb_badminton_best.pth.tar

=== grep repo for concrete WASB config numbers (288/512/frames_in=3 etc) ===

backend/pipeline/detectors/native_wasb_runner.py:272: argv += ["--frames_out_dir", frames_dir]

backend/pipeline/detectors/wasb_infer.py:151: frames_in: int,

backend/pipeline/detectors/wasb_infer.py:174: and manifest.get("frames_in") == frames_in

backend/pipeline/detectors/wasb_infer.py:257:def build_windows(frames: list, frames_in: int):

backend/pipeline/detectors/wasb_infer.py:258: """Sliding windows of ``frames\_in`` consecutive frame *paths* (the unit of GPU work

backend/pipeline/detectors/wasb_infer.py:261: Returns a list of windows, each a list of

```

``frames_in`` consecutive paths
backend/pipeline/detectors/wasb_infer.py:262: ([frames[i:i+frames_in] for i in
range(len(frames) - frames_in + 1)]). The
backend/pipeline/detectors/wasb_infer.py:266: return [frames[i : i + frames_in] for i in
range(len(frames) - frames_in + 1)]
backend/pipeline/detectors/wasb_infer.py:381: in a ``SequentialFrameStore`` (sliding buffer
sized to ``window`` = ``frames_in``); on a
backend/pipeline/detectors/wasb_infer.py:504: frames_in = int(cfg["model"]["frames_in"])
backend/pipeline/detectors/wasb_infer.py:505: input_wh = (int(cfg["model"]["inp_width"]),
int(cfg["model"]["inp_height"]))
backend/pipeline/detectors/wasb_infer.py:506: output_wh = (int(cfg["model"]["out_width"]),
int(cfg["model"]["out_height"]))
backend/pipeline/detectors/wasb_infer.py:524: if len(frames) < frames_in:
backend/pipeline/detectors/wasb_infer.py:526:     raise SystemExit(f"need >= {frames_in}")
frames, found {len(frames)} in {where}")
backend/pipeline/detectors/wasb_infer.py:528: # Sliding windows of ``frames_in`` consecutive
frames (the unit of GPU work + checkpoint).
backend/pipeline/detectors/wasb_infer.py:530: for window in build_windows(frames, frames_in):
backend/pipeline/detectors/wasb_infer.py:550:     frames_in=frames_in,
backend/pipeline/detectors/wasb_infer.py:570:     "frames_in": frames_in,
backend/pipeline/detectors/wasb_infer.py:630:         window=frames_in,
backend/pipeline/detectors/wasb_infer.py:842:     "--frames_out_dir", help="where to extract
frames when --video is used"
backend/pipeline/detectors/wasb_infer.py:912:         args.video, args.frames_out_dir or
(args.video + "_frames")
backend/pipeline/detectors/wasb_runner.py:177:         frame_args = f"--frames_out_dir
{wsl_tilde_quote(frames_dir)} "
docs/CODE_MAP.md:256:$ **manifest.json** (cache): {version, frames_dir(abspath), weights,
sport, frames_in, frames_total, windows_total, windows_done}`. Reuse only if ALL match +
`version==CACHE_VERSION`.
docs/CODE_MAP.md:271:- `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input
512x288 (WxH), ImageNet normalize; affine resize maps heatmap coords -> original-frame coords.
(model dims read from cfg["model"] at runtime in wasb_infer.run.)
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_2026-07-04.md:40:GPU; sliding
windows re-transform each frame ~`frames_in`x. At batch 64 the GPU is *still* ~1% ?
tests/test_native_wasb_runner.py:292: # Pin disk mode here (default is now streaming) so this
also covers the --frames_out_dir path.
tests/test_native_wasb_runner.py:307: assert "--frames_out_dir" in argv and "--stream-video"
not in argv
tests/test_native_wasb_runner.py:318: assert "--stream-video" in argv and "--frames_out_dir"
not in argv
tests/test_native_wasb_runner.py:356: assert argv[argv.index("--frames_out_dir") + 1] ==
expected_frames
tests/test_native_wasb_runner.py:365: assert argv[argv.index("--frames_out_dir") + 1] ==
expected_frames
tests/test_tracknet_runner.py:628: assert "--frames_out_dir" not in cmd # no PNG extraction
on the fast path
tests/test_tracknet_runner.py:635: assert "--frames_out_dir" in cmd
tests/test_wasb_cache.py:132: "frames_in": 3,
tests/test_wasb_cache.py:147: frames_in=3,
tests/test_wasb_cache.py:166: frames_in=3,
tests/test_wasb_cache.py:174: "frames_in": 3,
tests/test_wasb_cache.py:204: frames_in=3,
tests/test_wasb_cache.py:212: "frames_in": 3,
tests/test_wasb_cache.py:283: # frames_in=3 -> 4 sliding windows, each 3 consecutive,
overlapping by 2.

```

35. assistant (2026-07-06T20:14:21.038Z)

Excellent ? this is definitive. The checkpoint and `docs/CODE\_MAP.md:271` give me the exact WASB model config:

```

- `conv1.weight` = (64, **9**, 3, 3) ? 9 input channels = **frames_in=3 x 3 RGB** (frames
concatenated on channel dim)
- `final_layers.0.weight` = (**3**, 16, 1, 1) ? 3 output heatmaps (one per frame in window,
`frames_out=3`)

```

- CODE_MAP confirms: `name=hrnet, frames\_in=3, frames\_out=3, input 512x288 (WxH), ImageNet normalize`

Let me read the CODE_MAP WASB section and export_wasb_segments.py (which likely reveals the postprocess/detector internals since it exports segments), plus check config.json and eval_baselines/nightly_baseline.json for actual golden paths and baseline F1 numbers.

36. user (2026-07-06T20:14:23.048Z)

```
245     `{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server,
receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count,
shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare
`model_name`; motion adds random `intensity/avg_speed_kmh`.
246
247     $ **jobs row** (`db.create_job` / `get_job` ? #16, PR-1): `{id:TEXT(uuid4.hex),
video_path, video_id?:TEXT(set once hashed), segmenter?, player_pool?:JSON list, max_frames?,
status:'queued'|"running"|"done"|"failed' (EXECUTION state ? pipeline verdict is
result["status"]), progress?:TEXT(stage label), error?:TEXT, result?:JSON(the former sync
/api/process response incl. reports), created_at, started_at?, finished_at?}`.
248
249     $ **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time,
end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start;
`_candidate_id` stamped internally).
250
251     $ **GT / golden CSVs** (TWO conventions ? do not conflate):
252     - generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal
or MM:SS/HH:MM:SS; RAISES on bad row.
253     - golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
254
255     $ **det_raw.jsonl** (one window/line): `{ "w":<window_index>,
"f":[[frame_id,[x,y,score,scale],...], ...]}`. Lines MUST be contiguous (line N has `w==N`);
first violation/JSONDecodeError -> truncate-heal.
256     $ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
257     $ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv,
...}]}`; ? DB key = file MD5, manifest `video_id` is a label.
258
259     $ **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP),
unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives
,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.
260
261     $ **WSL detector candidate** (in-WSL, model<->tracker): `{ 'xy':np.array([x,y]),
'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST
rebuild `xy` as np.array (tracker does `np.linalg.norm`).
262
263     $ **HitEvent** (audio, `pipeline/audio/hit_detector`): `{t:float(sec), strength:float}`
? onset-envelope peak.
264
265     $ **observability report** (`reporting/harvest.record_run` -> `RunRecorder`): stages
`validation/segmentation/ai_handoff` + highlights + verdict ? {pass,pass_with_warnings,fail};
flags `{code,faq_ref}`; `video_meta.fps_source` ? {probed,fallback}. ? stage/metric/detail names
are coupled to `spec.yaml::rules[].when[].path` ? pairs that MUST stay in sync:
`ai_handoff.details.{api_key_present,ai_based}`, `segmentation.metrics.segment_count`, `segmenta
tion.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinges
t}`, `validation.status`, `video_meta.fps_source`.
266
267     ---
268
269     ## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ?
claims below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
270     - Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra
```

```

`compose(config_name='eval', overrides=[dataset=<sport>, model=wasb,
detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is
single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
271 - `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH),
ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read
from cfg["model"] at runtime in wasb_infer.run.)
272 - `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats)` -> (results,
hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.
(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
273 - `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes
every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()`
resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls
`build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y`
? consistent.)
274 - `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh,
transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)
275 - Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-
data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial,
avoided.
276
277 ---
278
279 ## 5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified
here)
280 - Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-
encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest
tracks the exact file used.
281 - `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv`
(indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` +
`@scripts/run_phase3_eval.py`.
282 - Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) ->
consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.
283
284 ---
285
286 ## 6. GOTCHAS (cross-cutting footguns)
287 - **MD5 keying:** `video_id` = MD5 of WHOLE file via the single
`@backend/utils/hashing.py::compute_video_id` helper (64KB chunks), imported by main + all
run_*.py. Manifest `video_id` is a label, NOT the DB key.
288 - **Config access:** `backend/main.py` and all four `run_*.py` load via
`backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains
for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in
`main.py` that only fires if `global_config/.indexing` is `None` (the ASGI-import edge case).
289 - **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `motion`
getattr fallback noted above.

```

37. user (2026-07-06T20:14:23.551Z)

```

1      {
2          "sport": "badminton",
3          "ai_provider": "gemini",
4          "ai_model": "gemini-flash-latest",
5          "validation": {
6              "min_resolution_width": 1280,
7              "min_resolution_height": 720,
8              "min_fps": 29.9,
9              "min_blur_threshold": 20.0,
10             "max_scene_cuts_per_minute": 0.5,
11             "relevance": {
12                 "court_green_lower": [

```

```

13             35,
14             40,
15             40
16         ],
17         "court_green_upper": [
18             85,
19             255,
20             255
21         ],
22         "court_pixels_min_ratio": 0.05
23     }
24 },
25     "indexing": {
26         "default_segmenter": "gemini",
27         "use_gpu": true,
28         "motion_threshold": 0.08,
29         "min_segment_duration": 3.0,
30         "dead_time_merge_gap": 2.0,
31         "chunk_duration": 90,
32         "chunk_overlap": 10,
33         "detector_model": "fasterrcnn_resnet50_fpn",
34         "detector_score_threshold": 0.5,
35         "yolo_velocity_threshold": 0.15,
36         "yolo_frame_skip": 3,
37         "yolo_tracker": "bytetrack.yaml",
38         "yolo_prefetch_workers": 2,
39         "min_action_window_duration": 2.0,
40         "log_yolo_candidates": true,
41         "store_yolo_candidates": true,
42         "ai_handoff_padding": 2.0,
43         "ai_max_workers": 1,
44         "tracknet": {
45             "wsl_distro": "Ubuntu",
46             "repo_dir": "~/models/TrackNetV4",
47             "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48             "conda_env": "TrackNetV4",
49             "weights_path": "",
50             "queue_length": 5,
51             "stage_in_wsl": true,
52             "wsl_stage_dir": "~/clips",
53             "timeout_sec": 1800,
54             "velocity_threshold": 0.02
55         },
56         "wasb": {
57             "wsl_distro": "Ubuntu",
58             "repo_dir": "~/models/WASB-SBDT",
59             "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60             "conda_env": "wasb",
61             "weights_path": "~/models/WASB-
62             SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
63             "sport": "badminton",
64             "wsl_stage_dir": "~/clips",
65             "timeout_sec": 1800,
66             "velocity_threshold": 0.02
67         }
68     },
69     "output": {
70         "default_highlights_name": "highlights.mp4",
71         "db_name": "sports_indexer.db"
72     },
73     "stitching": {
74         "begin_padding": 0.5,
75         "end_padding": 0.5,
76         "use_cache": false,
77         "min_shot_count": 1,

```

```

77         "watermark": {
78             "enabled": true,
79             "text": "Generated by Khelsutra.guru",
80             "position": "bottom-right",
81             "opacity": 0.85,
82             "logo_path": "",
83             "font_path": ""
84         }
85     }
86 }

```

38. user (2026-07-06T20:14:27.638Z)

```

=== nightly_baseline.json (F1 numbers) ===
/configs/baseline_off/aggregate/f1@0.25 = 0.39981006941991765
/configs/baseline_off/aggregate/f1@0.5 = 0.2366145803119009
/configs/baseline_off/aggregate/precision = 0.2577557018251538
/configs/baseline_off/aggregate/recall = 0.22719343607869802
/configs/baseline_off/aggregate/tol_f1 = 0.1382064040388172
/configs/baseline_off/aggregate/tol_precision = 0.14536686157033415
/configs/baseline_off/aggregate/tol_recall = 0.13445461315285676
/configs/baseline_off/iou = 0.5
/configs/baseline_off/per_video/Badminton_BXH_2/f1@0.25 = 0.0
/configs/baseline_off/per_video/Badminton_BXH_2/f1@0.5 = 0.0
/configs/baseline_off/per_video/Badminton_BXH_2/tol_f1 = 0.0
/configs/baseline_off/per_video/Boxhill_Doubles/f1@0.25 = 0.10526315789473685
/configs/baseline_off/per_video/Boxhill_Doubles/f1@0.5 = 0.03508771929824561
/configs/baseline_off/per_video/Boxhill_Doubles/tol_f1 = 0.0
/configs/baseline_off/per_video/GX010128/f1@0.25 = 0.14925373134328357
/configs/baseline_off/per_video/GX010128/f1@0.5 = 0.05970149253731343
/configs/baseline_off/per_video/GX010128/tol_f1 = 0.0
/configs/baseline_off/per_video/GX010137/f1@0.25 = 0.9253731343283583
/configs/baseline_off/per_video/GX010137/f1@0.5 = 0.5970149253731343
/configs/baseline_off/per_video/GX010137/tol_f1 = 0.4477611940298507
/configs/baseline_off/per_video/GX010141/f1@0.25 = 0.5416666666666666
/configs/baseline_off/per_video/GX010141/f1@0.5 = 0.3333333333333333
/configs/baseline_off/per_video/GX010141/tol_f1 = 0.3333333333333333
/configs/baseline_off/per_video/GX020094/f1@0.25 = 0.7333333333333334
/configs/baseline_off/per_video/GX020094/f1@0.5 = 0.4444444444444445
/configs/baseline_off/per_video/GX020094/tol_f1 = 0.2888888888888889
/configs/baseline_off/per_video/GX030094/f1@0.25 = 0.5352112676056339
/configs/baseline_off/per_video/GX030094/f1@0.5 = 0.28169014084507044
/configs/baseline_off/per_video/GX030094/tol_f1 = 0.14084507042253522
/configs/baseline_off/per_video/adarsh_avi_singles/f1@0.25 = 0.8601036269430052
/configs/baseline_off/per_video/adarsh_avi_singles/f1@0.5 = 0.6321243523316062
/configs/baseline_off/per_video/adarsh_avi_singles/tol_f1 = 0.38341968911917096
/configs/baseline_off/per_video/gbaaddy/f1@0.25 = 0.17142857142857143
/configs/baseline_off/per_video/gbaaddy/f1@0.5 = 0.08571428571428572
/configs/baseline_off/per_video/gbaaddy/tol_f1 = 0.0
/configs/baseline_off/per_video/kushagra_singles/f1@0.25 = 0.0
/configs/baseline_off/per_video/kushagra_singles/f1@0.5 = 0.0
/configs/baseline_off/per_video/kushagra_singles/tol_f1 = 0.0
/configs/baseline_off/per_video/largetest_doubles/f1@0.25 = 0.8118811881188118
/configs/baseline_off/per_video/largetest_doubles/f1@0.5 = 0.5346534653465348
/configs/baseline_off/per_video/largetest_doubles/tol_f1 = 0.297029702970297
/configs/baseline_off/per_video/mahadevpura_1/f1@0.25 = 0.0
/configs/baseline_off/per_video/mahadevpura_1/f1@0.5 = 0.0
/configs/baseline_off/per_video/mahadevpura_1/tol_f1 = 0.0
/configs/baseline_off/per_video/mahadevpura_2/f1@0.25 = 0.0
/configs/baseline_off/per_video/mahadevpura_2/f1@0.5 = 0.0
/configs/baseline_off/per_video/mahadevpura_2/tol_f1 = 0.0
/configs/baseline_off/per_video/mahadevpura_singles/f1@0.25 = 0.6181818181818182
/configs/baseline_off/per_video/mahadevpura_singles/f1@0.5 = 0.3636363636363636
/configs/baseline_off/per_video/mahadevpura_singles/tol_f1 = 0.1818181818181818
/configs/baseline_off/per_video/testlarge_short/f1@0.25 = 0.5454545454545454

```

```

/configs/baseline_off/per_video/testlarge_short/f1@0.5 = 0.1818181818181818
/configs/baseline_off/per_video/testlarge_short/tol_f1 = 0.0
/configs/baseline_off/preset/name = baseline_off
/configs/default/aggregate/f1@0.25 = 0.6351342900954481
/configs/default/aggregate/f1@0.5 = 0.47226205620233463
/configs/default/aggregate/precision = 0.4144811747149071
/configs/default/aggregate/recall = 0.5722200200034974
/configs/default/aggregate/tol_f1 = 0.36356483482541063
/configs/default/aggregate/tol_precision = 0.3193624497645106
/configs/default/aggregate/tol_recall = 0.44074854650051454
/configs/default/iou = 0.5
/configs/default/per_video/Badminton_BXH_2/f1@0.25 = 0.6557377049180326
/configs/default/per_video/Badminton_BXH_2/f1@0.5 = 0.459016393442623
/configs/default/per_video/Badminton_BXH_2/tol_f1 = 0.3442622950819672
/configs/default/per_video/Boxhill_Doubles/f1@0.25 = 0.5625
/configs/default/per_video/Boxhill_Doubles/f1@0.5 = 0.42187499999999994
/configs/default/per_video/Boxhill_Doubles/tol_f1 = 0.28125
/configs/default/per_video/GX010128/f1@0.25 = 0.7164179104477613
/configs/default/per_video/GX010128/f1@0.5 = 0.582089552238806
/configs/default/per_video/GX010128/tol_f1 = 0.47761194029850756
/configs/default/per_video/GX010137/f1@0.25 = 0.8571428571428571
/configs/default/per_video/GX010137/f1@0.5 = 0.7999999999999999
/configs/default/per_video/GX010137/tol_f1 = 0.6
/configs/default/per_video/GX010141/f1@0.25 = 0.7843137254901961
/configs/default/per_video/GX010141/f1@0.5 = 0.5490196078431373
/configs/default/per_video/GX010141/tol_f1 = 0.47058823529411764
/configs/default/per_video/GX020094/f1@0.25 = 0.6966292134831462
/configs/default/per_video/GX020094/f1@0.5 = 0.4719101123595506
/configs/default/per_video/GX020094/tol_f1 = 0.4044943820224719

```

39. assistant (2026-07-06T20:14:33.721Z)

Now I understand the full picture. The `nightly\_baseline.json` uses `f1@0.25`, `f1@0.5`, and `tol\_f1` ? these are **rally-boundary** metrics (start/end time IoU + tolerance match), NOT per-pixel shuttle F1. This is the `rally\_seg\_eval.py` harness. The tol_f1 here is time-tolerance (? in seconds), from tolerance_metrics.py.

Let me confirm the rally_seg_eval metric definitions and check the `rally\_seg\_eval.py` for the exact tol_f1/?, and look at the export_wasb_segments.py to see if there's ANY per-pixel shuttle scoring. Let me also verify the WASB normalization/postprocess claim isn't recorded anywhere else, and read the golden_manifest to find the video/label pairing and quick-run subset.

40. user (2026-07-06T20:14:37.762Z)

```

=== rally_seg_eval: tol_f1 / tau / f1@ definitions ===
6:IoU-F1 is blind to merged mega-windows, so we report it alongside segmental F1@k, the
41:#: as the default split point for the stratified report (`--long-tau` overrides it).
DURATION, not
44:LONG_RALLY_TAU = 5.0
85:     tau_start: Optional[float] = None,
86:     tau_end: Optional[float] = None,
92:     over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown) so a
95:     Also carries the R2 task-faithful block (`tolerance_metrics`, the headline going
forward;
97:     tolerance-match `tol_f1/tol_precision/tol_recall` (over-production costs precision) +
the
104:     from backend.eval import tolerance_metrics as tm
108:     ts = tm.TAU_START if tau_start is None else tau_start
109:     te = tm.TAU_END if tau_end is None else tau_end
113:     tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
120:         "precision": sc.precision,
121:         "recall": sc.recall,
125:         "f1@0.1": f1k[0.1],
126:         "f1@0.25": f1k[0.25],
127:         "f1@0.5": f1k[0.5],

```

```

137:         # --- R2 (tolerance-match) ? the task-faithful headline block ---
138:         "tau_start": ts,
139:         "tau_end": te,
140:         "tol_f1": tol_f1,
141:         "tol_precision": tol_p,
142:         "tol_recall": tol_r,
153:         "f1@0.25",
154:         "f1@0.5",
158:         "precision",
159:         "recall",
160:         "tol_f1",
161:         "tol_precision",
162:         "tol_recall",
298:         tau_start: Optional[float],
299:         tau_end: Optional[float],
305:         row = score_intervals(preds, gts, iou, tau_start, tau_end, min_duration)
317:         tau_start: Optional[float] = None,
318:         tau_end: Optional[float] = None,
322:         rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block, plus
328:         tau_start,
329:         tau_end,
349:         tau_start: Optional[float] = None,
350:         tau_end: Optional[float] = None,
357:         sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also reports ?
on the
363:         from backend.eval import tolerance_metrics as tm
366:         golden, base, iou, base_min_crossings, tau_start, tau_end, min_duration
369:         golden, cand, iou, cand_min_crossings, tau_start, tau_end, min_duration
376:         "f1@0.25",
377:         "tol_f1",
405:         tau_start: Optional[float] = None,
406:         tau_end: Optional[float] = None,
407:         long_tau: float = LONG_RALLY_TAU,
409:         """The long-rally lens payoff: ``{all vs long(>long_tau)} × {single vs multicourt}``
aggregates.

```

41. user (2026-07-06T20:14:39.761Z)

```

3:After calibration picks a tuned threshold config, this turns a cached WASB
106: preds: List[Interval], gts: List[Interval], iou_threshold: float = 0.5
110: mr = match_intervals(preds, gts, iou_threshold)
111: sc = score(preds, gts, iou_threshold, match_result=mr)

```

42. assistant (2026-07-06T20:14:49.511Z)

Confirmed. This nails down the eval side definitively:

```

- ``f1@0.25``, ``f1@0.5`` = segmental F1 at IoU-k on rally time intervals (start,end seconds)
- ``tol_f1`` = tolerance-match F1 from `tolerance_metrics.match_1to1` with `??` in SECONDS**
(?_start=2.0s, ?_end=1.5s), NOT pixels
- There is `no per-pixel shuttle-position F1/tolF1` anywhere in this repo. The shuttle
detector's (x,y) output is consumed only to produce rally windows, and F1 is measured at the
rally-boundary level.

```

The background agent is reading the golden-set docs. Let me now confirm the golden manifest structure (video?label pairing, quick-run subset) and check `scripts/nightly\_regression.py` / `rally\_seg\_eval.py` for the exact single-video eval command. Let me also verify the fps=59.94/frame_width=1920 defaults and check for the WASB normalization anywhere else.

43. user (2026-07-06T20:14:51.562Z)

```

1         """Portable golden-corpus manifest loader and smoke checker.
2

```

```

3     The owner-side golden manifest is intentionally gitignored and has historically
4     carried absolute paths from whichever box ingested a clip. This module keeps that
5     artifact usable after the workspace moves by resolving stale paths against the
6     current repo layout and failing loudly when required corpus files are absent.
7
8     Default local layout supported:
9     - ../Annotation Setup/Trajectories/<trajectory>.csv
10    - ../Annotation Setup/Golden Labelled/<labels>.csv
11    - output/<labels>.csv
12
13    Extra roots can be supplied with ``--root`` or ``RALLY_GOLDEN_SEARCH_ROOTS``.
14    ""
15
16    from __future__ import annotations
17
18    import argparse
19    import json
20    import os
21    import sys
22    from dataclasses import dataclass
23    from pathlib import Path, PureWindowsPath
24    from typing import Any, Iterable, Sequence
25
26    DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
27    SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"
28    DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
29
30    _FIELD_DIR = {
31        "trajectory": "Trajectories",
32        "labels": "Golden Labelled",
33    }
34
35
36    @dataclass(frozen=True)
37    class ManifestIssue:
38        """One manifest path problem found by the resolver."""
39
40        name: str
41        field: str
42        path: str
43        reason: str
44
45        def message(self) -> str:
46            shown = f" {self.path!r}" if self.path else ""
47            return f"{self.name}: {self.field}{shown} - {self.reason}"
48
49
50    @dataclass(frozen=True)
51    class ManifestRebase:
52        """One path field changed from a stale manifest value to a local file."""
53
54        name: str
55        field: str
56        old_path: str
57        new_path: str
58
59        def message(self) -> str:
60            return f"{self.name}: {self.field} {self.old_path!r} -> {self.new_path!r}"
61
62
63    class ManifestPathError(RuntimeError):
64        """Raised when a strict manifest load finds missing required files."""
65
66        def __init__(self, manifest_path: str | os.PathLike[str], issues:
67            Sequence[ManifestIssue]):

```

```

67         self.manifest_path = os.fspath(manifest_path)
68         self.issues = list(issues)
69         detail = "\n".join(f" - {i.message()}" for i in self.issues[:12])
70         more = "" if len(self.issues) <= 12 else f"\n ... {len(self.issues) - 12} more"
71         super().__init__(
72             f"golden manifest path check failed for {self.manifest_path}: "
73             f"{len(self.issues)} issue(s)\n{detail}{more}"
74         )
75
76
77     def repo_root() -> Path:
78         """Return the repository root for this module."""
79
80         return Path(__file__).resolve().parents[2]
81
82
83     def load_manifest(manifest_path: str | os.PathLike[str]) -> list[dict[str, Any]]:
84         """Load the flat golden manifest list and validate its outer shape."""
85
86         path = Path(manifest_path)
87         with path.open(encoding="utf-8") as f:
88             data = json.load(f)
89             if not isinstance(data, list):
90                 raise ValueError(f"golden manifest must be a JSON list: {path}")

```

44. user (2026-07-06T20:14:56.195Z)

```

=== find golden manifest json files ===
./artifacts/a5_floor_smoke/analysis-a5-smoke/manifest.json
./artifacts/rally_tal_gen0/0.1.0/manifest.json
./artifacts/rally_tal_gen0/0.1.1-n15-shadow.20260704/manifest.json
./eval_baselines/fixtures/manifest.json
./output/cloud_lovo_manifest.json
./output/human_lovo_manifest.json
./output/human_lovo_manifest.local.json
./output/local_calib_manifest.json
./output/wasb/Adarsh v Avi_bb961d0d0cff_wasb_wasbcache/manifest.json
./output/wasb/GX010125_1080p__89605e80ed01_wasb_wasbcache/manifest.json

```

```

=== a golden .rallies.csv sample (columns) ===
rally_number,start_time,end_time,ending_reason,sport,shots_count
1,9.661,32.173,forced_error,badminton,
2,40.419,47.165,unforced_error,badminton,
3,53.165,57.672,other,badminton,
4,65.180,73.675,unforced_error,badminton,
...

```

```

=== nightly_regression.py invocation of rally_seg_eval ===
3:Re-scores the **golden corpus** under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
216:     """Compact, comparable summary of one :func:`rally_seg_eval.evaluate` result.
629:     from backend.eval import rally_seg_eval
631:     golden = rally_seg_eval.load_golden(manifest, features_dir)
644:     result = rally_seg_eval.evaluate(
690:     p.add_argument("--manifest", default=DEFAULT_MANIFEST, help="golden manifest JSON")
737:def main(argv: Optional[List[str]] = None) -> int:

```

45. assistant (2026-07-06T20:14:59.562Z)

Now I have the golden manifest structure and the label format. Let me read the human_lovo_manifest.json to get the actual video?label?trajectory pairing and the quick-run subset, plus confirm the rally_seg_eval single-video eval entry point.

46. user (2026-07-06T20:15:04.346Z)

```

=== human_lovo_manifest.json (first 3 entries, keys+values) ===
n entries: 15
---
name: adarsh_avi_singles
trajectory: C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/adarsh_avi_full_traj.csv
fps: 59.94
frame_width: 1920
labels: C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/Adarsh and
Avi.rallies.csv
---
name: kushagra_singles
trajectory: C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/kushagra_traj.csv
fps: 59.94
frame_width: 1920
labels: C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/KushagraYashAviFirstVideo.rallies.csv
---
name: targettest_doubles
trajectory: C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/targettestvideo_traj.csv
fps: 29.97
frame_width: 1920
labels: C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/LargeTestVideo.rallies.csv
=== all names + fps + frame_width ===
adarsh_avi_singles: fps=59.94 fw=1920 labels=C:/Users/avidu/Projects/Annotation
Setup/Collect/Golden Labelled/Adarsh and Avi.rallies.csv
kushagra_singles: fps=59.94 fw=1920 labels=C:/Users/avidu/Projects/Annotation
Setup/Collect/Golden Labelled/KushagraYashAviFirstVideo.rallies.csv
targettest_doubles: fps=29.97 fw=1920 labels=C:/Users/avidu/Projects/Annotation
Setup/Collect/Golden Labelled/LargeTestVideo.rallies.csv
gbaaddy: fps=59.94 fw=1920 labels=C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/gBaaddy20May2026.rallies.csv
testlarge_short: fps=29.97 fw=1920 labels=C:/Users/avidu/Projects/Annotation
Setup/Collect/Golden Labelled/TestLargeVideo.rallies.csv
mahadevpura_singles: fps=29.97 fw=1920 labels=C:/Users/avidu/Projects/Annotation
Setup/Collect/Golden Labelled/MahadevpuraSingles.rallies.csv
mahadevpura_2: fps=59.94 fw=1920.0 labels=C:/Users/avidu/Projects/badminton-highlight-
indexer/output/mahadevpura_2.rallies.csv
GX010128: fps=59.94 fw=1920.0 labels=C:/Users/avidu/Projects/badminton-highlight-
indexer/output/GX010128.rallies.csv
mahadevpura_1: fps=59.94 fw=1920.0 labels=C:/Users/avidu/Projects/badminton-highlight-
indexer/output/mahadevpura_1.rallies.csv
GX030094: fps=29.97 fw=1920.0 labels=C:/Users/avidu/Projects/badminton-highlight-
indexer/output/GX030094.rallies.csv
Badminton_BXH_2: fps=59.94 fw=1920.0 labels=C:/Users/avidu/Projects/badminton-highlight-
indexer/output/Badminton_BXH_2.rallies.csv
GX010141: fps=59.94 fw=1920.0 labels=C:/Users/avidu/Projects/badminton-highlight-
indexer/output/GX010141.rallies.csv
GX010137: fps=59.94 fw=1920.0 labels=C:/Users/avidu/Projects/badminton-highlight-
indexer/output/GX010137.rallies.csv
Boxhill_Doubles: fps=29.97 fw=1920.0 labels=C:/Users/avidu/Projects/badminton-highlight-
indexer/output/Boxhill_Doubles.rallies.csv
GX020094: fps=29.97 fw=1920.0 labels=C:/Users/avidu/Projects/badminton-highlight-
indexer/output/GX020094.rallies.csv

```

47. user (2026-07-06T20:15:06.296Z)

```

=== rally_seg_eval single-video eval / CLI entry ===
24:import argparse
183:def load_golden(
312:def evaluate(
573:def _build_preset(args: argparse.Namespace, side: str) -> WindowingPreset:
623:def main() -> None:

```

```

624:     ap = argparse.ArgumentParser(
627:     ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
628:     ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
629:     ap.add_argument(
632:     ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
633:     ap.add_argument("--iou", type=float, default=DEFAULT_IOU)
634:     ap.add_argument(
640:     ap.add_argument(
646:     ap.add_argument(
652:     ap.add_argument(
659:     ap.add_argument(
665:     ap.add_argument(
670:     ap.add_argument(
676:     ap.add_argument(
681:     ap.add_argument(
687:     ap.add_argument(
693:     ap.add_argument(
699:     ap.add_argument(
705:     ap.add_argument(
711:     ap.add_argument(
717:     ap.add_argument(
723:     ap.add_argument(
729:     ap.add_argument(
735:     ap.add_argument(
741:     ap.add_argument(
747:     ap.add_argument(
753:     ap.add_argument(
762:     ap.add_argument(
769:     ap.add_argument(
776:     ap.add_argument(
847: if __name__ == "__main__":

```

48. assistant (2026-07-06T20:15:14.195Z)

I now have everything on the eval side. The manifest confirms: the harness is `**trajectory-CSV ? rally-window ? rally-boundary F1**`, keyed on ``.rallies.csv`` (rally_number,start_time,end_time,...) labels. It never scores per-pixel shuttle position. Let me read the `rally_seg_eval` CLI options (``--only``, ``--features-dir``) and the ``evaluate`/`load_golden`` signatures to give an exact single-video command, and confirm the ``tol_f1`` ? units one more time. Let me also check the R2 doc snippet region for the ? definition. The background agent covers docs, but let me grab the `rally_seg_eval` CLI precisely.

49. user (2026-07-06T20:15:15.644Z)

```

623     def main() -> None:
624         ap = argparse.ArgumentParser(
625             description="Rally-segmentation eval vs golden labels (offline)."
626         )
627         ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
628         ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
629         ap.add_argument(
630             "--preset", default=SERVED.name, help="windowing preset (default: served)"
631         )
632         ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
633         ap.add_argument("--iou", type=float, default=DEFAULT_IOU)
634         ap.add_argument(
635             "--min-crossings",
636             type=int,
637             default=0,
638             help="net-crossings rally-state gate on --preset (0=off; W2 Gate-A)",
639         )
640         ap.add_argument(
641             "--vs-min-crossings",
642             type=int,
643             default=0,

```

```

644         help="net-crossings gate on --vs (default: same as --min-crossings)",
645     )
646     ap.add_argument(
647         "--max-window-duration",
648         type=float,
649         default=None,
650         help="override bounded-windowing cap (s) on BOTH presets (W1; e.g. 12)",
651     )
652     ap.add_argument(
653         "--vs-max-window-duration",
654         type=float,
655         default=None,
656         help="cap (s) on --vs ONLY (default: same as --max-window-duration); lets a "
657         "BASELINE (no cap) vs CANDIDATE (capped) promotion check run in one shot",
658     )
659     ap.add_argument(
660         "--hard-cap",
661         action="store_true",
662         help="enforce the cap as a HARD cap on --preset (chop continuously-active "
663         "windows at the cap when no inactivity gap exists; W1 hard-cap fallback)",
664     )
665     ap.add_argument(
666         "--vs-hard-cap",
667         action="store_true",
668         help="hard-cap fallback on --vs (default: same as --hard-cap)",
669     )
670     ap.add_argument(
671         "--blob-fallback",
672         action="store_true",
673         help="R5 blob-fallback on --preset (after R4, force-chop any group still over
the "
674         "cap at its midpoint + flag/log it; the continuously-active blob last resort)",
675     )
676     ap.add_argument(
677         "--vs-blob-fallback",
678         action="store_true",
679         help="R5 blob-fallback on --vs (default: same as --blob-fallback)",
680     )
681     ap.add_argument(
682         "--blob-factor",
683         type=float,
684         default=None,
685         help="R5 magnitude gate (both presets): chop only groups > this × cap (default
4.0)",
686     )
687     ap.add_argument(
688         "--inactivity-end",
689         type=float,
690         default=None,
691         help="R4 rally-end inactivity trim (s) on --preset (0=off; trims post-rally
drift tail)",
692     )

```

50. assistant (2026-07-06T20:15:20.070Z)

I have enough to write the eval command. The `rally\_seg\_eval.py` operates on the whole manifest (with `--only` likely filtering, though I see it uses `--preset`/`--manifest`/`--features-dir`; let me confirm whether `--only` exists). Let me quickly check for a per-video filter and the `evaluate`/`load\_golden` signatures, then wait for the docs agent.

51. user (2026-07-06T20:15:25.060Z)

=== --only / filter in rally_seg_eval main ===

39:#: SHORT (motion blips, background-court fragments on multicourt footage); filtering to long rallies

```

42:#: shot-count, is the filter: golden ``shots_count`` is unlabeled and the no-AI path reports
it as
67:def filter_by_min_duration(ivs: List[Interval], min_duration: float) -> List[Interval]:
68:     """The long-rally lens: keep only intervals strictly longer than ``min_duration``
seconds.
71:     positive duration, so nothing is dropped). It's a FILTER, not a new metric: applied
identically
100:     ``min_duration`` > 0 applies the **long-rally lens** (:func:`filter_by_min_duration`):
BOTH preds
101:     and GTs are filtered to rallies longer than that many seconds BEFORE any matching, so
the whole
102:     row (tIoU set AND R2 block) scores only long rallies. Default 0.0 = OFF (exact
passthrough).
106:     preds = filter_by_min_duration(preds, min_duration)
107:     gts = filter_by_min_duration(gts, min_duration)

=== evaluate() + load_golden() signatures ===
def load_golden(
    manifest_path: str = DEFAULT_MANIFEST, features_dir: str = DEFAULT_FEATURES_DIR
) -> List[Dict[str, Any]]:
    """Load each golden video's trajectory path + fps + frame_width (manifest) and GT rally
intervals (``<name>_golden_features.json`` ``gts``). Videos with no GTs are kept but
flagged."""
    manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
    out: List[Dict[str, Any]] = []
    for v in manifest:
        feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
        gts: List[Interval] = []
        if os.path.exists(feat):
            with open(feat) as fh:
                gts = [
                    (float(s), float(e)) for s, e in (json.load(fh).get("gts") or [])
                ]
        entry: Dict[str, Any] = {
            "name": v["name"],
            "trajectory": v["trajectory"],
        }
def evaluate(
    golden: List[Dict[str, Any]],
    preset: WindowingPreset,
    iou: float = DEFAULT_IOU,
    min_crossings: int = 0,
    tau_start: Optional[float] = None,
    tau_end: Optional[float] = None,
    min_duration: float = 0.0,
) -> Dict[str, Any]:
    """Score every golden video under ``preset`` (+ optional net-crossings gate); return per-
video
rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match block,
plus
its ``court_type``. ``min_duration`` > 0 applies the long-rally lens (preds + GTs filtered
to
rallies longer than that many seconds) consistently across both metric paths."""
    rows = _score_predicted(
        predict_all(golden, preset, min_crossings),
        iou,
        tau_start,
        tau_end,
        min_duration,
    )
    return {
        "preset": preset.to_dict(),
        "iou": iou,
        "min_crossings": min_crossings,
    }

=== DEFAULT_FEATURES_DIR / DEFAULT_IOU / DEFAULT_MANIFEST ===

```

```

10:windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the SERVED
30:from backend.eval.golden_manifest import DEFAULT_MANIFEST, load_resolved_manifest
32:from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
34:DEFAULT_FEATURES_DIR = "output"
35:DEFAULT_IOU = 0.5
84:     iou: float = DEFAULT_IOU,
184:     manifest_path: str = DEFAULT_MANIFEST, features_dir: str = DEFAULT_FEATURES_DIR
315:     iou: float = DEFAULT_IOU,
346:     iou: float = DEFAULT_IOU,
403:     iou: float = DEFAULT_IOU,

```

52. assistant (2026-07-06T20:15:29.390Z)

I now have a complete picture. ``rally_seg_eval.evaluate`` scores the whole ``golden`` list (no per-video ``--only``; the loop is over the manifest). The GTs come from ``<name>_golden_features.json`` (the ``gts`` field), and predictions come from re-windowing the WASB trajectory CSV. Let me wait for the docs agent to complete, which will confirm the GCS locations and any doc-level `?/F1` definition, then compile the final report. Let me check on it.

53. user (2026-07-06T20:15:35.932Z)

```

=== nightly_baseline.json: default config aggregate + tau + config names ===
top keys: ['_doc', 'configs', 'generated_at', 'git_commit', 'iou', 'manifest', 'tolerances']
config presets: ['baseline_off', 'default', 'milestone']
--- baseline_off aggregate: f1@0.25=0.39981006941991765, f1@0.5=0.2366145803119009,
tol_f1=0.1382064040388172, tau_start=None, tau_end=None
--- default aggregate: f1@0.25=0.6351342900954481, f1@0.5=0.47226205620233463,
tol_f1=0.36356483482541063, tau_start=None, tau_end=None
--- milestone aggregate: f1@0.25=0.6351342900954481, f1@0.5=0.47226205620233463,
tol_f1=0.36356483482541063, tau_start=None, tau_end=None
generated_at = 2026-07-03

```

54. user (2026-07-06T20:15:43.380Z)

```

_doc: Frozen nightly rally-quality regression baseline. Regenerate with `python
scripts/nightly_regression.py --update-baseline` after an intended improvement or a corpus
change. Tied to the LOCAL golden corpus (not committed).
iou: 0.5
manifest: output/human_lovo_manifest.json
tolerances: {'metric_tol': 0.005, 'over_seg_tol': 0, 'rate_tol': 0.01}
git_commit: a68f02e

```

55. assistant (2026-07-06T20:15:46.861Z)

I have all the eval-side facts. The ``tolerances`` here (`metric_tol=0.005`) are the `**regression gate tolerances**` (how much a metric may drop), not the `?`. The `?` for `tol_f1` is in ``tolerance_metrics.py`` (`TAU_START=2.0s`, `TAU_END=1.5s`). Now let me wait for the docs agent for the GCS locations. Let me check its status.

56. user (2026-07-06T20:15:49.253Z)

waiting for docs agent

57. assistant (2026-07-06T20:17:17.084Z)

The docs agent has completed with comprehensive results. I now have everything to compile the final report. Let me do one final check to confirm the WASB normalization/postprocess details since the WASB-SBDT repo isn't cloned locally ? I'll verify what the repo-side code reveals vs. what must be read from upstream, and note it as an open item precisely.

I have all the evidence needed. Let me compile the final report.

WASB Reimplementation + Golden Eval ? Investigation Report

****Critical framing finding up front:**** This repo's F1/tolF1 harness scores ****rally time-boundary intervals**** (start/end seconds), NOT per-pixel shuttle position. There is ****no per-frame shuttle-position F1 (TrackNet-style within-?-pixels) anywhere in this repo.**** The WASB detector emits a per-frame trajectory CSV; that feeds rally windowing; F1/tolF1 are measured on the resulting rally intervals. If you want a per-pixel shuttle metric to validate your reimplementation, you'll have to build it (or diff trajectory CSVs directly). See §2.

Also: the WASB-SBDT model repo (`~/models/WASB-SBDT`) is an ****external MIT dependency, not cloned locally**** (`C:/Users/avidu/models/WASB-SBDT` is empty). The repo-side caller (`wasb\_infer.py`) reads all model dims from Hydra config at runtime, so exact normalization constants live upstream. Where I can't confirm from local code, I inspected the actual weights file and flag the rest as open.

1. WASB inference internals

All in `backend/pipeline/detectors/wasb\_infer.py`. The runner `native\_wasb\_runner.py` is just a subprocess wrapper that stages this file into `~/models/WASB-SBDT/src/` and invokes it.

Model construction + weight load

- Config is composed via ****Hydra**** (`\_build\_cfg`, `wasb\_infer.py:57-78`):

```
```python
compose(config_name="eval", overrides=[
 f"dataset={sport}", # badminton
 "model=wasb", # -> name=hrnet
 f"detector.model_path={weights}",
 f"runner.device={device}", # cuda default
 f"runner.gpus={gpus}", # [0] on cuda
])
```
```
- The model is built by ****WASB's own `build\_detector(cfg)`**** (`wasb\_infer.py:499, 601`) ? HRNet construction and the `torch.load`/`load\_state\_dict` happen ****inside WASB's `detectors/detector.py::TracknetV2Detector`****, which is NOT in this repo (confirmed `docs/CODE\_MAP.md:269-275`). So you must replicate HRNet + the load from the upstream repo.
- ****Weight file structure ? verified directly**** from `scratch/stage/wasb\_badminton\_best.pth.tar`:
 - `torch.load(...)` returns a dict with a ****single key: `model\_state\_dict`**** (428 entries).
 - So the load is `model.load\_state\_dict(torch.load(path, map\_location=...)[`model_state_dict`])`.
 - First keys: `conv1.weight, bn1.{weight,bias,running\_mean,running\_var,num\_batches\_tracked}, conv2?` ? a standard HRNet stem.
 - ****`conv1.weight` shape = `(64, 9, 3, 3)`**** ? ****9 input channels = 3 frames × 3 RGB**** stacked on the channel axis.
 - ****`final\_layers.0.weight` shape = `(3, 16, 1, 1)`**, ****`final\_layers.0.bias` = `(3,)`**** ? ****3 output heatmaps**** (one per input frame; `frames\_out=3`), 1×1 conv head from 16 feature channels.

Preprocessing (the exact numbers you need)

Read from `cfg["model"]` at runtime (`wasb\_infer.py:504-506`):

```
```python
frames_in = int(cfg["model"]["frames_in"]) # = 3
input_wh = (cfg["model"]["inp_width"], cfg["model"]["inp_height"]) # = (512, 288)
WxH
output_wh = (cfg["model"]["out_width"], cfg["model"]["out_height"]) # = (512, 288)
```
```

- ****Input resolution: 512×288 (W×H)**** ? confirmed `docs/CODE\_MAP.md:271`: ****`frames\_in=3, frames\_out=3, input 512x288 (WxH), ImageNet normalize`****.
- ****Normalization: ImageNet mean/std**** (`CODE\_MAP.md:271`). The exact constants ([0.485,0.456,0.406]/[0.229,0.224,0.225]) are applied inside WASB's `build\_img\_transforms(cfg)` (`wasb\_infer.py:487, 581`) ? ****not in this repo; confirm the exact values + whether a `/255` scale precedes it from upstream `dataloaders`**** ? open.
- ****Color order: RGB.**** The disk path is `Image.open(path).convert('RGB')` (WASB's `read\_image`,

```
`wasb_infer.py:370-373`); the streaming path deliberately reproduces it byte-for-byte via
`cv2.cvtColor(bgr, COLOR_BGR2RGB)` ? PIL RGB (`_bgr_to_pil_rgb`, `wasb_infer.py:328-340`).
- Windowing / stacking: sliding windows of 3 consecutive frames, stride 1
(`build_windows`, `wasb_infer.py:257-266`): `[frames[i:i+3] for i in range(N-2)]`. Each window ?
sample `{ "frames": [3 paths], "annos": [Center(is_visible=False, x=-1, y=-1) per frame] }`
(`wasb_infer.py:529-535`). The 3 frames are concatenated on the channel dim ? 9-channel input
inside WASB's `ImageDataset` (`wasb_infer.py:586-594`). One window = one unit of GPU work.
```

Forward + postprocess

```
- Forward: detector.run_tensor(imgs, trans) -> (batch_results, hms_vis)
(`wasb_infer.py:646`). `trans` = the affine transform matrices that map heatmap coords back to
original-frame pixels. The heatmap-decode (argmax/blob/threshold) happens inside WASB's
`run_tensor` (upstream, not in repo) ? CODE_MAP.md:272 describes the contract:
`results[bid][eid]` = list of candidate dicts {xy: np.array (ORIGINAL image coords), score,
scale}. ? The actual decode algorithm (it's a blob/connected-component detector with a score
threshold and multi-scale in WASB, not a plain argmax) must be read from upstream
`detectors/detector.py`.
- Per-frame candidate dicts are accumulated by frame id across all windows the frame appears in
(`det_by_fid`, `wasb_infer.py:648-656`) and cached to `det_raw.jsonl` as [x,y,score,scale]
(`_det_plain`, `wasb_infer.py:112-115`).
- Trajectory from candidates: WASB's stateful `OnlineTracker` (`build_tracker(cfg)`,
`wasb_infer.py:685-690`; CODE_MAP.md:273). `tracker.refresh()` then
`tracker.update(frame_dets)` per frame in strict order ? {x, y, visi, score}. It does
`np.linalg.norm` vector math, so `xy` must be a real numpy array (`_dets_to_tracker`,
`wasb_infer.py:118-126`). Deterministic on a full ordered re-run. The tracker logic is also
upstream. ?
```

Output format

```
`_atomic_write_trajectory` (`wasb_infer.py:228-237`, rows built at `687-692`) emits a CSV with
header Frame,Visibility,X,Y, one row per frame:
- Frame = integer frame id; Visibility = 1 if tracker says visible else 0; X,Y = tracked
shuttle pixel coords in original-frame resolution (or -1,-1 when not visible).
- This is exactly the shape TrackNetRunner.parse_trajectory_csv` consumes (model-agnostic;
shared with TrackNet).
```

2. Golden eval harness ? how F1/tolF1 is computed

There are two interval metrics; both are rally-time-boundary, ? is in SECONDS, not pixels.

(A) tIoU-F1 ? the current gate. `run\_eval.py` ? `backend/eval/harness.py`:
- A prediction matches a GT rally when **temporal IoU (overlap/union) ? threshold**, greedy 1:1.
Default **IoU ? = 0.5** (`run\_eval.py:73`). $P = TP / (TP + FP)$, $R = TP / (TP + FN)$, $F1 = \text{harmonic mean}$
(`docs/EVALUATION.md`).
- Scores predictions **already in the SQLite DB** ? it does NOT run the pipeline.

(B) tolF1 / F1@? ? the R2 tolerance metric (`backend/eval/tolerance\_metrics.py`, reported
alongside tIoU by `rally\_seg\_eval.py`):
- A prediction matches a golden rally iff `start ? ?_start` AND `end ? ?_end`` under a
strict 1:1 Hungarian assignment (scipy `linear\_sum\_assignment`; greedy fallback if no scipy)
? `tolerance\_metrics.py:34-119`.
- **values (seconds):** `TAU_START = 2.0`, `TAU_END = 1.5` (`tolerance\_metrics.py:29-31`);
symmetric sweep `{1.5, 2.0, 2.5, 3.0}` (`SWEEP\_TAU\_S`, line 31). Asymmetric because the
service window makes rally-**start** fuzzy. These are provisional pending an IAA study
(`docs/R2\_EVAL\_METRIC\_DESIGN.md`).
- `tol_f1 = matched * 2 * P * R / (P + R)`, `tol_precision = matched / n_pred`, `tol_recall = matched / n_gt`
(`tolerance\_metrics.py:98-119`, emitted by `rally\_seg\_eval.score\_intervals` at `140-142`).
Relationship: `tolF1@-3.0s ? tIoU-F1@0.5`.

"tolF1 vs F1": F1 (a.k.a. `f1@0.25`/`f1@0.5`) = overlap-IoU-gated match at that IoU. tolF1 =
boundary-tolerance-gated match within ? seconds. Both are rally-level.

Commands

```

- **DB-based single video (tIoU-F1):**
  ...

  python run_eval.py --video "C:/path/match.mp4" --annotations rallies.csv --stage both --iou
0.5
  # or: python run_eval.py --video-id <md5> --annotations rallies.csv --iou 0.5
  ...

  Needs: the video **already processed** into `output/sports_indexer.db` + the GT `rallies.csv`.
- **Trajectory-CSV based, full golden corpus (both f1@k and tol_f1, no DB, no GPU):** this is
the one that actually exercises the WASB output end-to-end:
  ...

  python -m backend.eval.rally_seg_eval --manifest output/human_lovo_manifest.json --features-
dir output --iou 0.5
  ...

  `rally_seg_eval.evaluate` (`rally_seg_eval.py:312`) loops the manifest, re-windows each
trajectory CSV, and scores vs `_golden_features.json` `gts`. There is **no per-video
`--only`** here; it scores the whole manifest. Optional `--tau-start/--tau-end` override the R2
?.
- **Per-video rally calibration from a single trajectory CSV** (closest to a single-clip numbers
run):
  ...

  python -m backend.eval.calibrate_wasb --trajectory full_traj.csv --labels "X.rallies.csv"
--fps 59.94 --frame-width 1920
  ...

  (`calibrate_wasb.py:131-143`)

```

3. Golden video + ground-truth label locations & format

****Manifest (source of truth for pairing):**** `output/human\_lovo\_manifest.json` ? a 15-entry JSON list, each entry `{name, trajectory, fps, frame\_width, labels}`. GCS mirror: `gs://khelsutra-rally-corpus/manifests/golden.json`.

****Videos:****

```

- GCS contract layout: `gs://khelsutra-rally-corpus/videos/<name>.mp4`,
`.../labels/<date><name>.rallies.csv`, `.../trajectories/<name>.csv` (`docs/DATA_IN_GCS.md`).
**? Reality: only 1/15 videos actually migrated** (`mahadevpura_1`); the rest live locally on
Drive `G:\My Drive\...\Golden Label Videos Request - June 15\` and the `F:\[Khelsutra] GoPro...`
archive.
- The trajectory CSVs (the GPU-free eval input ? you don't need the raw videos to run
`rally_seg_eval`) are referenced in the manifest at `C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/<...>.csv`.

```

****Labels (ground truth):****

```

- Format: `.rallies.csv` with columns
**`rally_number,start_time,end_time,ending_reason,sport,shots_count`** (decimal seconds).
Verified sample (`scratch/stage/GX010128.rallies.csv`):
  ...

```

```

  rally_number,start_time,end_time,ending_reason,sport,shots_count
  1,9.661,32.173,forced_error,badminton,
  ...

```

```

- The canonical loader `backend/eval/gt_loader.py` reads **only start/end** (accepts
`start`/`start_time` + `end`/`end_time` headers, or positional cols 0,1; ignores all other
columns). It does **not** contain per-frame shuttle x,y ? these are rally intervals, confirming
there is no per-pixel GT here.

```

```

- **Pairing:** by `name` ? `videos/<name>.mp4` ? `labels/<name>.rallies.csv` ?
`trajectories/<name>.csv`, all joined via the manifest `name`. GCS labels carry a `YYYY-MM-DD_`
prefix that's stripped at join time. Strong key is the file MD5
(`backend/utils/hashing.py::compute_video_id`).

```

****Quick-run subset (2-3 clips), all 59.94 fps, frame_width 1920:****

```

name	fps	rallies	video_id	notes
`GX010128`	59.94	52	`db84f3e65318`	has a worked tolF1 preview

```

```
| `mahadevpura_2` | 59.94 | 40 | `038e0e0915a2` | primary R2 preview clip |
| `mahadevpura_1` | 59.94 | 10 | `38bef75d78cb` | smallest; the only clip already in GCS
`videos/` |
```

? **Per-clip pixel resolution is not documented** (only spec target 1080p). ``ffprobe`` the actual proxy if you need exact WxH. The label ``rallies.csv`` files for these are staged locally in ``scratch/stage/`` (e.g. ``GX010128.rallies.csv``, ``mahadevpura-2_proxy.rallies.csv``) and mirrored in ``output/``.

4. Baseline numbers to compare against

``eval_baselines/nightly_baseline.json`` (generated 2026-07-03, git_commit ``a68f02e``, ``iou=0.5``, ``manifest=output/human_lovo_manifest.json``). It carries three presets. **The ``default`` (= ``milestone``) preset aggregate is your target:**

```
preset	f1@0.25	f1@0.5	tol_f1	tol_precision	tol_recall
`default` / `milestone`	**0.6351**	**0.4723**	**0.3636**	0.3194	0.4407
`baseline_off`	0.3998	0.2366	0.1382	0.1454	0.1345
```

Per-video ``default``-preset numbers are in the same file (``configs.default.per_video.<name>.{f1@0.25,f1@0.5,tol_f1}``), e.g. sanity-check anchors:

- ``GX010137``: `f1@0.5 = 0.800`, `tol_f1 = 0.600` (best clip)
- ``GX010128``: `f1@0.5 = 0.582`, `tol_f1 = 0.478`
- ``adarsh_avi_singles``: `f1@0.5 = 0.632` (`baseline_off`), and default per-video available
- Several clips (``mahadevpura_1/2``, ``kushagra_singles``) score 0.0 at `baseline_off`.

Regression gate tolerances (how far a metric may drop before it fails) are separate: ``tolerances = {metric_tol: 0.005, over_seg_tol: 0, rate_tol: 0.01}``.

Important caveat for your reimplementation: these baselines score the **rally-windowed output**, not raw shuttle detections. They confound (a) detector accuracy with (b) the windowing preset. To validate a torch-2.x WASB reimplementation **specifically**, the cleaner check is: run both the reference ``wasb_infer.py`` and your reimplementation on the same clip and **diff** the ``Frame,Visibility,X,Y`` trajectory CSVs directly (sub-pixel float tolerance) ? this is exactly the parity gate ``native_wasb_runner.py:26-34`` documents (validated 2026-06-20: 1194/1195 frames byte-identical on a fixed GPU, lone diff ~5e-5 px). Feeding both CSVs through ``rally_seg_eval`` and comparing the F1 table is the downstream sanity check.

Open questions (flagged)

1. **Exact ImageNet normalization constants + any ``/255`` prescale, and the heatmap-decode algorithm (blob vs argmax, score threshold, multi-scale), and the OnlineTracker logic** are all in the **upstream WASB-SBDT repo, not cloned locally** (``~/models/WASB-SBDT`` is empty). Clone ``nttcom/WASB-SBDT`` and read ``src/dataloaders/``, ``src/detectors/detector.py``, ``src/trackers/online.py``, and ``src/configs/model/wasb.yaml`` + ``src/configs/dataset/badminton.yaml`` to get them exactly. Repo-side code confirms input 512x288, frames_in=3, 9-channel input, 3-channel heatmap head, RGB, ImageNet-normalize.
2. **No per-pixel shuttle-position F1/tolF1 exists in this repo** ? the golden metric is rally-time-boundary. If your goal (b) is per-frame shuttle scoring, you'll need per-frame GT (the ``rallies.csv`` files don't have it) and a new metric.
3. **Per-clip resolution undocumented** ? ``ffprobe`` the proxies.
4. **14/15 golden videos not in GCS** ? use local Drive/F: proxies; for a numbers run, the trajectory CSVs referenced in ``output/human_lovo_manifest.json`` are sufficient (no video decode needed) via ``rally_seg_eval``.